DATA SCIENCE & MACHINE LEARNING COMPREHENSIVE TEXTBOOK

Master Reference Handbook: NumPy, Pandas, EDA & Machine Learning

Complete In-Depth Conceptual Foundations, Eye-Comfort Design, Method-by-Method API Breakdowns, Task Walkthroughs, Figure Creation Architecture & 7 Rendered Graphics

Pedagogical Architecture & Eye-Comfort Design:
This publication reference textbook is engineered to provide complete mastery across the modern Python data science stack. It incorporates an **ergonomic, eye-friendly color palette** (soft light-slate code boxes, soothing mint/sky callouts, and generous line spacing) designed specifically to prevent visual fatigue and eye strain during intensive study sessions.

Textbook Structure (Strictly 4 Core Sections + Reference Appendix):
• **Section 1: NumPy Complete Mastery (Modules 01-17)**: Contiguous C-memory layouts, SIMD vectorization, memory strides, data types, pre-initialized matrices, reshaping, views vs copies, broadcasting rules, ufuncs, multi-axis reductions, subgrid slicing, fancy indexing, multi-axis sorting, argmax/argmin, np.where, np.select, and linear algebra.
• **Section 2: Pandas Comprehensive Data Manipulation & Relational Analysis**: Series & DataFrame architecture, metadata inspection, loc/iloc/at/iat indexing, query strings, boolean masks, string operations (.str), missing data theory (MCAR/MAR) & imputation, deduplication, sorting, ranking, mutations, apply/map/transform, GroupBy split-apply-combine, pivot tables, cross-tabulations, relational merges (Inner, Left, Right, Full Outer), rolling window calculations, and complete walkthroughs for all 10 repository projects (including Set 2 PDF and 50 Join Task questions).
• **Section 3: Exploratory Data Analysis & Complete Figure Creation Guide**: EDA philosophy, Tukey's 5-stage lifecycle, IQR outlier detection, correlation matrices, plus the **Visual Figure Creation Architecture** (Figure vs Axes, DPI, figsize, spines removal, data value labels, tight_layout) and **6 high-resolution embedded rendered figures**.
• **Section 4: Machine Learning Foundations & Supervised Algorithms**: ML taxonomy, preprocessing pipeline, data leakage prevention, StandardScaler standardization, K-Nearest Neighbors (KNN) with Euclidean/Manhattan distance, probability foundations, Bayes' theorem derivation, Gaussian Naive Bayes architecture, model evaluation metrics (Accuracy, Precision, Recall, F1), and the **embedded Confusion Matrix Heatmap figure**.
• **Section 5: Master API Quick Reference Tables & Troubleshooting Guide**: NumPy/Pandas API tables and Top 10 Data Science Pitfalls with solutions.

Section	Core Curriculum & Modules	Key Demonstration Focus
Section 1 NumPy Mastery	17 Modules: Arrays, Contiguous Buffers, Special Matrices, Reshaping, Views vs Copies, Vectorization, Broadcasting Rules, Slicing, Multi-Axis Sorting, Argmax, np.where, Linear Algebra	C-speed memory buffers, broadcasting mechanics, multi-axis aggregations, advanced 2D sorting.
Section 2 Pandas Mastery	Comprehensive Methods: Series & DataFrame, head/tail/info/describe, loc/iloc/at/iat, query/isin/between, .str methods, isna/fillna/dropna/drop_duplicates, sort_values/rank/nlargest, apply/map/transform, pivot_table/crosstab, cumsum/rolling/shift, pd.merge/concat, 10 Repository Tasks	Exhaustive method-by-method breakdown: data cleaning, indexing, string manipulation, split-apply-combine, window functions, and joins.

Section 3 EDA & Figure Creation	EDA 5-Stage Lifecycle, Quality Audit, Correlation Matrices, **Visual Figure Creation Architecture** (Figure vs Axes, styling, annotations, tight_layout) + **6 Embedded Charts** (Bar, Horizontal Bar, Histogram, Pie, Scatter, Heatmap)	Step-by-step visual chart construction, aesthetic parameter tuning, real embedded high-res graphics.
Section 4 Machine Learning	ML Taxonomy, Preprocessing Pipeline, KNN Classifier & Euclidean Distance, Probability Theory, Bayes Theorem, Model Evaluation & Confusion Matrix Heatmap	Supervised learning pipeline, instance-based classification, probabilistic modeling, embedded Confusion Matrix chart.
Section 5 Reference Tables	Master NumPy & Pandas API Syntax Reference Tables and Top 10 Data Science Pitfalls with Solutions	Quick lookup syntax tables, anti-patterns, SettingWithCopyWarning fix, data leakage prevention.

SECTION 1: NumPy Complete Mastery (Modules 01-17 & Advanced Foundations)

Formal Definition: NumPy Array Architecture (ndarray)

The NumPy ndarray is an N-dimensional homogeneous array of fixed-size items. Unlike standard Python lists that store pointers to scattered objects across memory, an ndarray occupies a single contiguous memory block. This memory locality enables modern CPU architectures to perform SIMD (Single Instruction, Multiple Data) vectorized parallel computations, boosting execution speeds by 10x to 100x.

1.1 Array Architecture, Contiguous Memory Buffers & SIMD Acceleration

To truly understand NumPy's computational dominance, one must examine how standard Python lists differ from NumPy arrays at the hardware level. A standard Python list `[1, 2, 3]` is an array of *pointers* pointing to independent, fragmented Python integer objects scattered across RAM. Each individual integer object carries 28 bytes of metadata overhead (type pointer, reference count, and integer value). Consequently, when a Python loop executes, CPU cache misses occur repeatedly because memory addresses are fragmented.

In contrast, a NumPy array allocates a single contiguous block of raw binary bytes. For example, an `int64` array stores consecutive 8-byte integers directly adjacent in memory. This continuous locality allows modern CPUs to load whole cache lines (64 bytes) into L1 and L2 hardware caches and execute SIMD (Single Instruction, Multiple Data) instructions, processing 4 to 8 elements in parallel in a single clock cycle.

Fundamental Metadata Attributes of `ndarray`:

• .ndim: Integer indicating the number of axes/dimensions (1 = Vector, 2 = Matrix, 3 = Tensor).
• .shape: Tuple of integers indicating the length of the array along each dimension.
• .size: Total count of elements stored in the array (equal to the product of shape components).
• .dtype: The data type of stored elements (`int32`, `int64`, `float32`, `float64`).
• .itemsize: Memory consumed by a single element in bytes (e.g., 8 bytes for `int64`).
• .nbytes: Total memory consumed by the entire array in bytes (`.size * .itemsize`).
• .strides: Tuple of bytes to step in each dimension when traversing the array.

PYTHON IMPLEMENTATION

```
import numpy as np
# 1D Vector, 2D Matrix, and 3D Tensor Creation
v = np.array([1, 2, 3, 4], dtype=np.int64)
m = np.array([[1, 2, 3, 4], [5, 6, 7, 8]], dtype=np.int64)
t = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]], dtype=np.float32)
print('Vector -> ndim:', v.ndim, '| shape:', v.shape, '| size:', v.size, '| bytes:', v.nbytes, '| strides:', v.s
trides)
print('Matrix -> ndim:', m.ndim, '| shape:', m.shape, '| size:', m.size, '| bytes:', m.nbytes, '| strides:', m.s
trides)
print('Tensor -> ndim:', t.ndim, '| shape:', t.shape, '| itemsize:', t.itemsize, '| dtype:', t.dtype)
```

CONSOLE OUTPUT RESULT

```
Vector -> ndim: 1 | shape: (4,) | size: 4 | bytes: 32 | strides: (8,)
Matrix -> ndim: 2 | shape: (2, 4) | size: 8 | bytes: 64 | strides: (32, 8)
Tensor -> ndim: 3 | shape: (2, 2, 2) | itemsize: 4 | dtype: float32
```

1.2 Data Types, Bit Widths & Memory Overflow Mechanics

NumPy provides precise control over data representation. Choosing the optimal bit width reduces memory footprint drastically (e.g. converting `float64` to `float32` halves RAM consumption in deep learning models):

- int8: Signed 8-bit integer (-128 to 127). Consumes 1 byte.

- uint8: Unsigned 8-bit integer (0 to 255). Standard for image pixel channels.
- int32 / int64: Standard signed integers for indexing and counts.
- float32 / float64: Single (32-bit) and double (64-bit) precision IEEE 754 floating point.
- bool_: Boolean flag (True or False), stored as 1 byte.

Integer Overflow Warning

When an arithmetic operation exceeds the maximum representable value of a data type, integer overflow occurs without an exception. For instance, in int8, 127 + 1 overflows and wraps around to -128! Always verify numerical ranges before downcasting with astype().

PYTHON IMPLEMENTATION

```
# Integer overflow demonstration
x_int8 = np.array([126, 127], dtype=np.int8)
print('Initial int8 array:', x_int8)
print('Overflow (127 + 1 in int8):', x_int8 + np.int8(1))
# Safe type casting with .astype()
x_safe = x_int8.astype(np.int32) + 1
print('Safe casting to int32:', x_safe, '| dtype:', x_safe.dtype)
```

CONSOLE OUTPUT RESULT

```
Initial int8 array: [126 127]
Overflow (127 + 1 in int8): [ 127 -128]
Safe casting to int32: [127 128] | dtype: int32
```

1.3 Pre-initialized & Special Matrix Generators

When initializing network weights, masking image channels, or setting up identity matrices, manual entry is inefficient. NumPy provides specialized generator routines that allocate contiguous memory directly:

- np.zeros(shape, dtype): Allocates an array populated entirely with zeros.
- np.ones(shape, dtype): Allocates an array populated entirely with ones.
- np.full(shape, fill_value, dtype): Populates every element with a specified constant scalar.
- np.identity(n) / np.eye(N, M, k): Generates an identity matrix with ones on diagonal k and zeros elsewhere.
- np.diag(v): Constructs a diagonal matrix from 1D vector v, or extracts the diagonal from a 2D matrix.
- np.triu(m) / np.tril(m): Extracts upper or lower triangular elements of matrix m.
- np.empty(shape): Allocates an uninitialized memory buffer without clearing values (fastest initialization when values are immediately overwritten).

PYTHON IMPLEMENTATION

```
zeros = np.zeros((2, 3), dtype=int)
ones  = np.ones((2, 3), dtype=int)
full  = np.full((2, 3), fill_value=7, dtype=int)
eye   = np.eye(3, dtype=int)
diag_mat = np.diag([10, 20, 30])
tri_upper = np.triu([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print('Zeros (2x3):\n', zeros)
print('Full (2x3 filled with 7):\n', full)
print('Diagonal Matrix from Vector:\n', diag_mat)
print('Upper Triangular Matrix (triu):\n', tri_upper)
```

CONSOLE OUTPUT RESULT

```
Zeros (2x3):\n [[0 0 0]\n  [0 0 0]]
Full (2x3 filled with 7):\n [[7 7 7]\n  [7 7 7]]
Diagonal Matrix from Vector:\n [[10  0  0]\n  [ 0 20  0]\n  [ 0  0 30]]
Upper Triangular Matrix (triu):\n [[1 2 3]\n  [0 5 6]\n  [0 0 9]]
```

1.4 Numerical Sequences: `arange` vs. `linspace` vs. `logspace` & 2D Grids

Choosing Between arange, linspace, and logspace

Use np.arange(start, stop, step) when the step size between points is known (stop is exclusive). Use np.linspace(start, stop, num) when the exact count of evenly spaced samples is required (stop is inclusive). For floating-point steps, linspace is strictly preferred because arange suffers from floating-point accumulation errors. Use np.logspace(start, stop, num) for exponential scales (e.g., hyperparameter search).

PYTHON IMPLEMENTATION

```
# Step-based sequence: arange(start, stop, step)
s_arange = np.arange(0, 10, 2)           # [0, 2, 4, 6, 8] - stop is exclusive
# Sample-count sequence: linspace(start, stop, num_points)
s_lin = np.linspace(0, 1, 5)             # 5 evenly spaced points between 0 and 1
# Logarithmic sequence: 10^0 to 10^3
s_log = np.logspace(0, 3, 4)             # [1, 10, 100, 1000]
# 2D Coordinate Grid generation for surface modeling
x_coords = np.linspace(-1, 1, 3)
y_coords = np.linspace(-1, 1, 3)
X, Y = np.meshgrid(x_coords, y_coords)
print('np.arange(0, 10, 2):', s_arange)
print('np.linspace(0, 1, 5):', s_lin)
print('np.logspace(0, 3, 4):', s_log)
print('2D Meshgrid X coordinates:\n', X)
```

CONSOLE OUTPUT RESULT

```
np.arange(0, 10, 2): [0 2 4 6 8]
np.linspace(0, 1, 5): [0.   0.25 0.5  0.75 1.  ]
np.logspace(0, 3, 4): [   1.   10.  100. 1000.]
2D Meshgrid X coordinates:\n [[-1.  0.  1.]\n  [-1.  0.  1.]\n  [-1.  0.  1.]]
```

1.5 Reshaping, Transposing & Dimension Manipulation

Reshaping Mathematical Constraints

Reshaping alters how dimensions are indexed without modifying memory bytes. The product of dimensions in the new shape must strictly equal the total number of elements: prod(new_shape) == arr.size. You can specify -1 for at most one dimension to let NumPy automatically infer its size.

PYTHON IMPLEMENTATION

```
seq = np.arange(1, 13)                   # 12 elements: [1, 2, ..., 12]
mat_3x4 = seq.reshape((3, 4))            # Reshape to 3 rows x 4 cols
mat_auto = seq.reshape((2, -1))          # -1 infers 6 columns automatically
transposed = mat_3x4.T                   # Matrix transpose: (3, 4) -> (4, 3)
expanded = np.expand_dims(seq, axis=0)   # Shape (1, 12) - adding batch dimension
squeezed = np.squeeze(expanded)          # Shape (12,) - removing singleton axis
print('Reshaped (3x4):\n', mat_3x4)
print('Inferred Reshape (2, -1):\n', mat_auto)
print('Transposed Shape (.T):', transposed.shape)
print('Expanded Shape (expand_dims):', expanded.shape)
print('Squeezed Shape (squeeze):', squeezed.shape)
```

CONSOLE OUTPUT RESULT

```
Reshaped (3x4):\n [[ 1  2  3  4]\n  [ 5  6  7  8]\n  [ 9 10 11 12]]
Inferred Reshape (2, -1):\n [[ 1  2  3  4  5  6]\n  [ 7  8  9 10 11 12]]
Transposed Shape (.T): (4, 3)
Expanded Shape (expand_dims): (1, 12)
Squeezed Shape (squeeze): (12,)
```

1.6 Memory Views vs. Deep Copies (`flatten` vs. `ravel`)

Critical Distinction: Views Mutate Originals!

A view shares the underlying data buffer with the original array. Mutating an element inside a view mutates the original array! In contrast, a deep copy allocates an independent buffer. array.flatten() always returns a deep copy, whereas np.ravel() returns a memory-efficient view whenever possible.

PYTHON IMPLEMENTATION

```
base_arr = np.array([10, 20, 30, 40])
view_arr = base_arr.view()
copy_arr = base_arr.copy()
view_arr[0] = 999  # Mutates base_arr!
copy_arr[1] = 888  # Does NOT mutate base_arr
flat_copy = base_arr.flatten()  # Always deep copy
ravel_view = np.ravel(base_arr)  # View if memory contiguous
print('Original after view mutation:', base_arr)
print('Independent copy:', copy_arr)
print('view_arr.base is base_arr:', view_arr.base is base_arr)
print('flat_copy.base is None:', flat_copy.base is None)
print('ravel_view.base is base_arr:', ravel_view.base is base_arr)
```

CONSOLE OUTPUT RESULT

```
Original after view mutation: [999  20  30  40]
Independent copy: [ 10 888  30  40]
view_arr.base is base_arr: True
flat_copy.base is None: True
ravel_view.base is base_arr: True
```

1.7 Memory Strides & Layout (C-Contiguous vs. Fortran Order)

In a C-contiguous array (row-major order), consecutive elements of a row are adjacent in memory. In a Fortran-contiguous array (column-major order), consecutive elements of a column are adjacent in memory. The .strides attribute indicates how many bytes the pointer must advance to move to the next item along each axis:

PYTHON IMPLEMENTATION

```
arr_c = np.arange(6, dtype=np.int32).reshape(2, 3)  # C order (row-major)
arr_f = np.asfortranarray(arr_c)                    # Fortran order (col-major)
print('C-order strides (bytes per row, bytes per col):', arr_c.strides)
print('Fortran-order strides (bytes per row, bytes per col):', arr_f.strides)
print('arr_c C_CONTIGUOUS flag:', arr_c.flags['C_CONTIGUOUS'])
print('arr_c.T C_CONTIGUOUS flag:', arr_c.T.flags['C_CONTIGUOUS'])
```

CONSOLE OUTPUT RESULT

```
C-order strides (bytes per row, bytes per col): (12, 4)
Fortran-order strides (bytes per row, bytes per col): (4, 8)
arr_c C_CONTIGUOUS flag: True
arr_c.T C_CONTIGUOUS flag: False
```

1.8 Vectorized Arithmetic Operators & Scalar Broadcasting Mechanics

The Two Mathematical Rules of NumPy Broadcasting

When operating on arrays of differing shapes, NumPy compares their dimensions backward starting from the trailing (rightmost) dimension. Two dimensions are compatible when: (1) they are equal in length, OR (2) one of them is 1. If both conditions fail, a ValueError: operands could not be broadcast together is raised.

PYTHON IMPLEMENTATION

```
a = np.array([[1, 2, 3], [4, 5, 6]])      # Shape: (2, 3)
b = np.array([[2, 2, 2], [3, 3, 3]])      # Shape: (2, 3)
row_vec = np.array([10, 20, 30])          # Shape: (3,) -> Broadcasts to (2, 3)
col_vec = np.array([[100], [200]])        # Shape: (2, 1) -> Broadcasts to (2, 3)
# Outer product via broadcasting
u = np.array([1, 2, 3])
v = np.array([10, 20, 30])
outer_prod = u[:, np.newaxis] * v[np.newaxis, :]
print('Element-wise Addition (a + b):\n', a + b)
print('Broadcasting Row Vector (a + row_vec):\n', a + row_vec)
print('Broadcasting Column Vector (a + col_vec):\n', a + col_vec)
print('Outer Product Matrix via Broadcasting:\n', outer_prod)
```

CONSOLE OUTPUT RESULT

```
Element-wise Addition (a + b):\n [[ 3  4  5]\n  [ 7  8  9]]
Broadcasting Row Vector (a + row_vec):\n [[11 22 33]\n  [14 25 36]]
Broadcasting Column Vector (a + col_vec):\n [[101 102 103]\n  [204 205 206]]
Outer Product Matrix via Broadcasting:\n [[10 20 30]\n  [20 40 60]\n  [30 60 90]]
```

1.9 Universal Mathematical Functions (`ufuncs`) & In-Place Performance

NumPy Universal Functions (`ufuncs`) are compiled C routines operating element-wise across arrays. They completely avoid Python bytecode interpreter overhead and support accumulator arguments (`out=...`) for zero-allocation memory reuse:

PYTHON IMPLEMENTATION

```
vals = np.array([1, 4, 9, 16, 25], dtype=np.float64)
out_buffer = np.empty_like(vals)
np.sqrt(vals, out=out_buffer)  # In-place execution into out_buffer
print('np.sqrt(vals) via out_buffer:', out_buffer)
print('np.power(vals, 2):', np.power(vals, 2))
print('np.log(vals):', np.round(np.log(vals), 2))
# Ufunc reduction and accumulation
arr = np.array([1, 2, 3, 4])
print('np.add.reduce(arr):', np.add.reduce(arr))
print('np.multiply.accumulate(arr):', np.multiply.accumulate(arr))
```

CONSOLE OUTPUT RESULT

```
np.sqrt(vals) via out_buffer: [1. 2. 3. 4. 5.]
np.power(vals, 2): [  1.  16.  81. 256. 625.]
np.log(vals): [0.   1.39 2.2  2.77 3.22]
np.add.reduce(arr): 10
np.multiply.accumulate(arr): [ 1  2  6 24]
```

1.10 Multi-Axis Statistical Reductions & Aggregations

Multidimensional reductions collapse specific dimensions:

- axis=None: Aggregates across the entire array into a single global scalar.
- axis=0: Collapses vertically down columns; calculates metrics for each column.
- axis=1: Collapses horizontally across rows; calculates metrics along each row.
- keepdims=True: Preserves collapsed dimensions as size-1 axes, enabling immediate broadcasting back to original shapes (crucial for feature standardization).

PYTHON IMPLEMENTATION

```
m = np.array([[10, 20, 30], [40, 50, 60]])
print('Matrix:\n', m)
print('Global Sum (axis=None):', np.sum(m))
print('Column-wise Sum (axis=0):', np.sum(m, axis=0))
print('Row-wise Sum (axis=1):', np.sum(m, axis=1))
# Standardizing columns with keepdims=True
col_mean = np.mean(m, axis=0, keepdims=True)
col_std  = np.std(m, axis=0, keepdims=True)
standardized_m = (m - col_mean) / col_std
print('col_mean shape (keepdims=True):', col_mean.shape)
print('Standardized Matrix:\n', np.round(standardized_m, 2))
```

CONSOLE OUTPUT RESULT

```
Matrix:\n [[10 20 30]\n  [40 50 60]]
Global Sum (axis=None): 210
Column-wise Sum (axis=0): [50 70 90]
Row-wise Sum (axis=1): [ 60 150]
col_mean shape (keepdims=True): (1, 3)
Standardized Matrix:\n [[-1. -1. -1.]\n  [ 1.  1.  1.]]
```

1.11 Multi-Dimensional Slicing, Striding & 3D Tensor Manipulation

Slicing higher dimensional arrays and computer vision tensors (such as RGB images structured as `(height, width, channels)`):

PYTHON IMPLEMENTATION

```
grid = np.arange(1, 21).reshape(5, 4)
print('Base 5x4 Grid:\n', grid)
print('\nSub-grid grid[1:4, 1:3]:\n', grid[1:4, 1:3])
print('\nExtract Row 2 (grid[2, :]):', grid[2, :])
print('Extract Column 1 (grid[:, 1]):', grid[:, 1])
# 3D Tensor color channel extraction simulation: shape (2, 2, 3)
img_tensor = np.arange(12).reshape(2, 2, 3)
red_channel = img_tensor[:, :, 0]  # Extract red channel
print('\nSimulated RGB Tensor (2x2x3) Red Channel:\n', red_channel)
```

CONSOLE OUTPUT RESULT

```
Base 5x4 Grid:\n [[ 1  2  3  4]\n  [ 5  6  7  8]\n  [ 9 10 11 12]\n  [13 14 15 16]\n  [17 18 19 20]]
\nSub-grid grid[1:4, 1:3]:\n [[ 6  7]\n  [10 11]\n  [14 15]]
\nExtract Row 2 (grid[2, :]): [ 9 10 11 12]
Extract Column 1 (grid[:, 1]): [ 2  6 10 14 18]
\nSimulated RGB Tensor (2x2x3) Red Channel:\n [[0 3]\n  [6 9]]
```

1.12 Boolean Masking & Logical Filtering (`&`, `|`, `~`, `any`, `all`)

Bitwise operators filter arrays conditionally. Standard Python operators (`and`, `or`, `not`) cannot be used because they evaluate the truth of entire arrays rather than element-wise flags:

PYTHON IMPLEMENTATION

```
vals = np.array([10, 25, 30, 45, 50, 65, 70])
mask = (vals % 20 == 0) | (vals > 50)
print('Boolean Mask:\n', mask)
print('Masked Selection:', vals[mask])
print('np.any(vals > 60):', np.any(vals > 60))
print('np.all(vals > 0) :', np.all(vals > 0))
print('np.isin(vals, [25, 50, 100]):', np.isin(vals, [25, 50, 100]))
```

CONSOLE OUTPUT RESULT

```
Boolean Mask:\n [False False False False False  True  True]
Masked Selection: [65 70]
np.any(vals > 60): True
np.all(vals > 0) : True
np.isin(vals, [25, 50, 100]): [False  True False False  True False False]
```

1.13 Fancy Indexing & Integer Array Selection

Fancy Indexing vs. Slicing

Slicing returns a view into the array. In contrast, integer fancy indexing (passing an array of integer index positions) and boolean masking always allocate and return a deep copy.

PYTHON IMPLEMENTATION

```
matrix = np.arange(10, 100, 10).reshape(3, 3)
print('3x3 Matrix:\n', matrix)
# Paired coordinates: [row_idx], [col_idx] -> extracts (0, 0), (1, 2), (2, 1)
paired = matrix[[0, 1, 2], [0, 2, 1]]
# Rectangular sub-block using np.ix_
block = matrix[np.ix_([0, 2], [0, 2])]
print('Paired Coordinate Elements:', paired)
print('Rectangular 2x2 Sub-block (np.ix_):\n', block)
```

CONSOLE OUTPUT RESULT

```
3x3 Matrix:\n [[10 20 30]\n  [40 50 60]\n  [70 80 90]]
Paired Coordinate Elements: [10 60 80]
Rectangular 2x2 Sub-block (np.ix_):\n [[10 30]\n  [70 90]]
```

1.14 Sorting, Multi-Axis Sorting, Argsort & Lexsort

- np.sort(a): Returns sorted array values.
- np.argsort(a): Returns index positions that would sort the array.
- np.lexsort((secondary, primary)): Performs multi-key indirect sorting (first by primary, then by secondary).

PYTHON IMPLEMENTATION

```
vals = np.array([25, 10, 55, 30, 45])
print('Ascending Sort:', np.sort(vals))
print('Descending Sort (via [::-1]):', np.sort(vals)[::-1])
sorted_idx = np.argsort(vals)
print('Argsort Indices:', sorted_idx)
# Multi-key lexicographical sort: Sort students by Grade (descending), then Age (ascending)
grades = np.array([85, 92, 85, 78, 92])
ages   = np.array([22, 21, 20, 23, 24])
# lexsort accepts keys in reverse priority: secondary first, primary last
lex_idx = np.lexsort((ages, -grades))
print('Lexsort Indices (Grade desc, Age asc):', lex_idx)
```

CONSOLE OUTPUT RESULT

```
Ascending Sort: [10 25 30 45 55]
Descending Sort (via [::-1]): [55 45 30 25 10]
Argsort Indices: [1 0 3 4 2]
Lexsort Indices (Grade desc, Age asc): [1 4 2 0 3]
```

1.15 Extreme Value Indexing & Binary Search (`argmax`, `argmin`, `searchsorted`)

PYTHON IMPLEMENTATION

```
grid = np.array([[12, 45, 23], [98, 34, 67], [15, 89, 54]])
flat_max_idx = np.argmax(grid)
max_coords = np.unravel_index(flat_max_idx, grid.shape)
print('Matrix:\n', grid)
print('Global Argmax Flat Index:', flat_max_idx)
print(f'2D Matrix Coordinates of Max Value: Row {max_coords[0]}, Col {max_coords[1]}')
# Fast binary search insertion points with np.searchsorted
sorted_thresholds = np.array([0, 50, 75, 90, 100])
test_scores = np.array([45, 62, 88, 95])
bins = np.searchsorted(sorted_thresholds, test_scores)
print('Searchsorted Bin Positions for test_scores:', bins)
```

CONSOLE OUTPUT RESULT

```
Matrix:\n [[12 45 23]\n  [98 34 67]\n  [15 89 54]]
Global Argmax Flat Index: 3
2D Matrix Coordinates of Max Value: Row 1, Col 0
Searchsorted Bin Positions for test_scores: [1 2 3 4]
```

1.16 Conditional Filtering & Decisions (`np.where`, `np.select`, `clip`)

np.where and np.select Operational Modes

np.where(condition, [x, y]) operates in two modes: (1) Condition only: returns index tuples where condition is True. (2) Three arguments: performs element-wise ternary substitution (if condition yields x, else y). For multi-tier ladder decisions, np.select(condition_list, choice_list, default) handles complex business rules cleanly.

PYTHON IMPLEMENTATION

```
scores = np.array([45, 78, 92, 35, 60, 88])
# Mode 1: Search matching indices
passing_indices = np.where(scores >= 60)[0]
# Mode 2: Ternary replacement
status = np.where(scores >= 60, 'Pass', 'Fail')
# Mode 3: Multi-tier grading with np.select
conds = [scores >= 90, scores >= 75, scores >= 60]
grades = ['A', 'B', 'C']
letter_grades = np.select(conds, grades, default='F')
# Mode 4: Outlier bounding with np.clip
bounded_scores = np.clip(scores, 50, 85)
print('Passing Indices:', passing_indices)
print('Ternary Replacement:\n', status)
print('Multi-Tier Grading (np.select):\n', letter_grades)
print('Clipped Scores [50, 85]:', bounded_scores)
```

CONSOLE OUTPUT RESULT

```
Passing Indices: [1 2 4 5]
Ternary Replacement:\n ['Fail' 'Pass' 'Pass' 'Fail' 'Pass' 'Pass']
Multi-Tier Grading (np.select):\n ['F' 'B' 'A' 'F' 'C' 'B']
Clipped Scores [50, 85]: [50 78 85 50 60 85]
```

1.17 Linear Algebra Foundations: Matrix Decomposition, SVD & Eig

Linear algebra operations form the mathematical foundation for regression weights, neural activations, and covariance matrices:

PYTHON IMPLEMENTATION

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
dot_prod = np.dot(A, B)
matmul_prod = A @ B  # Matrix multiplication operator
det_A = np.linalg.det(A)
inv_A = np.linalg.inv(A)
# Eigenvalues and Eigenvectors
eigenvals, eigenvecs = np.linalg.eig(A)
# Singular Value Decomposition (SVD)
U, S, Vt = np.linalg.svd(A)
# Solving linear equations: 2x + y = 8, x + 3y = 13
coeffs = np.array([[2, 1], [1, 3]])
constants = np.array([8, 13])
solution = np.linalg.solve(coeffs, constants)
print('Matrix Multiplication (A @ B):\n', matmul_prod)
print(f'Determinant of A: {det_A:.2f}')
print('Eigenvalues:', np.round(eigenvals, 2))
print('Singular Values (SVD):', np.round(S, 2))
print(f'Linear System Solution -> x={solution[0]:.1f}, y={solution[1]:.1f}')
```

CONSOLE OUTPUT RESULT

```
Matrix Multiplication (A @ B):\n [[19 22]\n  [43 50]]
Determinant of A: -2.00
Eigenvalues: [-0.37  5.37]
Singular Values (SVD): [5.46 0.37]
Linear System Solution -> x=2.2, y=3.6
```

1.18 Structured Arrays: C-Style Structs in NumPy Contiguous Memory

NumPy structured arrays allow storing heterogeneous data fields within a single contiguous memory buffer (equivalent to C structs):

PYTHON IMPLEMENTATION

```
# Define compound data type: name (10-char string), age (int32), salary (float64)
emp_dtype = np.dtype([('name', 'U10'), ('age', 'i4'), ('salary', 'f8')])
records = np.array([('Akhil', 28, 75000.0), ('Sneha', 32, 88000.0)], dtype=emp_dtype)
print('Structured Array:', records)
print('Access by Field Name (salary):', records['salary'])
print('Access Record 0:', records[0])
```

CONSOLE OUTPUT RESULT

```
Structured Array: [('Akhil', 28, 75000.) ('Sneha', 32, 88000.)]
Access by Field Name (salary): [75000. 88000.]
Access Record 0: ('Akhil', 28, 75000.)
```

1.19 Advanced Array Stacking, Concatenation & Splitting

Combining and partitioning arrays along existing or new dimensions:

PYTHON IMPLEMENTATION

```
a1 = np.array([1, 2, 3])
a2 = np.array([4, 5, 6])
vstacked = np.vstack([a1, a2])     # Vertical stacking (rows)
hstacked = np.hstack([a1, a2])     # Horizontal stacking (1D append)
colstacked = np.column_stack([a1, a2]) # 2D column stacking
print('Vertical Stacking (vstack):\n', vstacked)
print('Horizontal Stacking (hstack):', hstacked)
print('Column Stacking (column_stack):\n', colstacked)
# Splitting 2D array back into pieces
top, bottom = np.vsplit(vstacked, 2)
print('Top row after vsplit:', top)
```

CONSOLE OUTPUT RESULT

```
Vertical Stacking (vstack):\n [[1 2 3]\n  [4 5 6]]
Horizontal Stacking (hstack): [1 2 3 4 5 6]
Column Stacking (column_stack):\n [[1 4]\n  [2 5]\n  [3 6]]
Top row after vsplit: [[1 2 3]]
```

1.20 Polynomial Fitting & Numerical Calculus (`polyfit`, `gradient`)

Least-squares polynomial curve fitting and finite-difference numerical gradients:

PYTHON IMPLEMENTATION

```
x_pts = np.array([0.0, 1.0, 2.0, 3.0, 4.0])
y_pts = np.array([1.0, 2.8, 5.1, 8.9, 14.2])
# Quadratic polynomial fit: y = ax^2 + bx + c
coeffs = np.polyfit(x_pts, y_pts, deg=2)
p = np.poly1d(coeffs)
# Numerical gradient of y_pts
grad = np.gradient(y_pts, x_pts)
print('Fitted Polynomial Equation:\n', p)
print('Numerical Gradients dy/dx at points:', np.round(grad, 2))
```

CONSOLE OUTPUT RESULT

```
Fitted Polynomial Equation:\n        2\n0.8071 x + 0.08143 x + 0.9886
Numerical Gradients dy/dx at points: [1.8  2.05 3.05 4.55 5.3 ]
```

1.21 Vectorized String Operations (`np.char`)

NumPy provides specialized C-accelerated string functions in the `np.char` namespace for performing element-wise string transformations on arrays of strings or bytes:

PYTHON IMPLEMENTATION

```
names = np.array(['  Alice  ', 'BOB', 'charlie', 'David_123'])
clean_names = np.char.strip(names)
upper_names = np.char.upper(clean_names)
has_a = np.char.count(upper_names, 'A')
replaced = np.char.replace(clean_names, '123', 'Senior')
print('Cleaned:', clean_names)
print('Uppercase:', upper_names)
print('Count of A:', has_a)
print('Replaced:', replaced)
```

CONSOLE OUTPUT RESULT

```
Cleaned: ['Alice' 'BOB' 'charlie' 'David_123']
Uppercase: ['ALICE' 'BOB' 'CHARLIE' 'DAVID_123']
Count of A: [1 0 1 1]
Replaced: ['Alice' 'BOB' 'charlie' 'David_Senior']
```

1.22 Modern Pseudo-Random Number Generation (`np.random.default_rng`)

Modern NumPy deprecates legacy `np.random.seed()` in favor of explicit `np.random.default_rng(seed)`. This utilizes the high-period PCG64 pseudo-random generator, providing reproducible stochastic distributions for Monte Carlo simulations and synthetic benchmarking:

PYTHON IMPLEMENTATION

```
rng = np.random.default_rng(seed=42)
# Normal (Gaussian) distribution: mu=10.0, sigma=2.0
normal_samples = rng.normal(loc=10.0, scale=2.0, size=5)
# Uniform continuous distribution: [0.0, 1.0)
uniform_samples = rng.uniform(low=0.0, high=1.0, size=5)
# Binomial discrete distribution: n=10 trials, p=0.5 success probability
binomial_samples = rng.binomial(n=10, p=0.5, size=5)
print('Normal (mu=10, sigma=2):', np.round(normal_samples, 2))
print('Uniform [0, 1):', np.round(uniform_samples, 2))
print('Binomial (n=10, p=0.5):', binomial_samples)
```

CONSOLE OUTPUT RESULT

```
Normal (mu=10, sigma=2): [10.61  7.92 11.5  11.88  6.1 ]
Uniform [0, 1): [0.98 0.76 0.79 0.13 0.45]
Binomial (n=10, p=0.5): [4 7 6 6 5]
```

1.23 Set Operations & Array Membership Testing (`unique`, `intersect1d`, `setdiff1d`)

Performing relational set logic on 1D arrays without converting to Python sets preserves contiguous memory locality and vectorization speed:

PYTHON IMPLEMENTATION

```
arr1 = np.array([10, 20, 30, 40, 50, 20, 30])
arr2 = np.array([30, 40, 50, 60, 70])
u_vals, u_counts = np.unique(arr1, return_counts=True)
intersection = np.intersect1d(arr1, arr2)
difference = np.setdiff1d(arr1, arr2)
union = np.union1d(arr1, arr2)
print('Unique values:', u_vals, '| counts:', u_counts)
print('Intersection:', intersection)
print('Difference (arr1 - arr2):', difference)
print('Union:', union)
```

CONSOLE OUTPUT RESULT

```
Unique values: [10 20 30 40 50] | counts: [1 2 2 1 1]
Intersection: [30 40 50]
Difference (arr1 - arr2): [10 20]
Union: [10 20 30 40 50 60 70]
```

1.24 Multi-Dimensional Array Iteration (`np.nditer`)

When element-wise iteration is strictly necessary, `np.nditer` traverses multi-dimensional arrays respecting underlying memory layout (C-order or Fortran-order) without constructing expensive intermediate temporary arrays:

PYTHON IMPLEMENTATION

```
matrix = np.arange(6).reshape(2, 3)
iter_vals = []
# Traverse in optimal C-contiguous memory order
for val in np.nditer(matrix):
    iter_vals.append(int(val))
# In-place mutation with readwrite op_flags
mut_matrix = matrix.copy()
for x in np.nditer(mut_matrix, op_flags=['readwrite']):
    x[...] = 2 * x
print('Original Matrix:\n', matrix)
print('Traversed values via nditer:', iter_vals)
print('Mutated 2x Matrix via nditer:\n', mut_matrix)
```

CONSOLE OUTPUT RESULT

```
Original Matrix:\n [[0 1 2]\n  [3 4 5]]
Traversed values via nditer: [0, 1, 2, 3, 4, 5]
Mutated 2x Matrix via nditer:\n [[ 0  2  4]\n  [ 6  8 10]]
```

SECTION 2: Pandas Comprehensive Data Manipulation & Relational Analysis

Formal Definition: Pandas Data Structures (Series & DataFrames)

Pandas is an open-source library providing high-performance, easy-to-use data structures and data analysis tools for Python. A pd.Series is a 1-dimensional labeled array with an index. A pd.DataFrame is a 2-dimensional labeled data structure with columns of potentially different types, functioning conceptually like an in-memory SQL database table or spreadsheet.

2.1 Core Data Structures: Series vs. DataFrame Construction

Pandas structures can be instantiated from Python dictionaries, lists, NumPy arrays, or external files:

PYTHON IMPLEMENTATION

```
import pandas as pd
# 1. Series from list and dictionary
s_list = pd.Series([10, 20, 30, 40], name='Scores')
s_dict = pd.Series({'a': 100, 'b': 200, 'c': 300})
# 2. Vectorized series arithmetic
s1 = pd.Series([1, 2, 3]); s2 = pd.Series([10, 20, 30])
print('s1.add(s2):\n', s1.add(s2))
print('s1.multiply(s2):\n', s1.multiply(s2))
```

CONSOLE OUTPUT RESULT

```
s1.add(s2):\n 0    11\n 1    22\n 2    33\n dtype: int64
s1.multiply(s2):\n 0    10\n 1    40\n 2    90\n dtype: int64
```

2.2 Dataset Inspection & Metadata Auditing Methods

Before performing data cleaning or modeling, inspect structural attributes and memory footprints:

- df.head(n) / df.tail(n): Preview first/last n rows.
- df.sample(n, random_state=42): Extracts an unbiased random sample of records.
- df.shape: Tuple of (rows, columns).
- df.info(): Reports column names, non-null counts, datatypes, and RAM consumption.
- df.describe(): Computes statistical summaries (count, mean, std, quartiles, min, max).
- df.memory_usage(deep=True): Detailed per-column memory usage in bytes.

PYTHON IMPLEMENTATION

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Evan', 'Fiona'],
    'Dept': ['IT', 'HR', 'IT', 'Finance', 'HR', 'IT'],
    'Salary': [72000, 54000, 88000, 95000, 61000, 82000],
    'Experience': [3, 2, 6, 8, 4, 5],
    'City': ['New York', 'London', 'New York', 'Paris', 'London', 'Tokyo']
})
print('Shape:', df.shape)
print('Columns:', df.columns.tolist())
print('Memory Usage (bytes):\n', df.memory_usage(deep=True))
print('\nSummary Stats:\n', df.describe()[['Salary', 'Experience']])
```

CONSOLE OUTPUT RESULT

```
Shape: (6, 5)
Columns: ['Name', 'Dept', 'Salary', 'Experience', 'City']
Memory Usage (bytes):\n Index         132\n Name          372\n Dept          360\n Salary         48\n Experien
ce     48\n City          384\n dtype: int64
\nSummary Stats:\n              Salary  Experience\ncount       6.00000    6.000000\nmean    75333.33333    4.66
6667\nstd     15845.08336    2.160247\nmin     54000.00000    2.000000\nmax     95000.00000    8.000000
```

2.3 Selection, Slicing & Indexing Methods (`loc`, `iloc`, `at`, `iat`)

Critical Distinction: loc vs. iloc vs. at vs. iat

df.loc[row_names, col_names] accesses elements by exact index labels (INCLUSIVE endpoint). df.iloc[row_pos, col_pos] accesses elements by integer coordinates (EXCLUSIVE endpoint). df.at and df.iat provide ultra-fast scalar access to single cell values. Never use chained indexing df[col][row] = val as it triggers SettingWithCopyWarning!

PYTHON IMPLEMENTATION

```
df_idx = df.set_index('Name')
# 1. loc with explicit labels
print('loc (Alice to Charlie, Salary & Dept):\n', df_idx.loc['Alice':'Charlie', ['Dept', 'Salary']])
# 2. iloc with integer positions
print('\niloc (rows 0:2, cols 0:2):\n', df_idx.iloc[0:2, 0:2])
# 3. Fast scalar cell access
print('\nat["Alice", "Salary"]:', df_idx.at['Alice', 'Salary'], '| iat[0, 1]:', df_idx.iat[0, 1])
```

CONSOLE OUTPUT RESULT

```
loc (Alice to Charlie, Salary & Dept):\n          Dept  Salary\nName                 \nAlice      IT   72000\nB
ob        HR   54000\nCharlie    IT   88000
\niloc (rows 0:2, cols 0:2):\n         Dept  Salary\nName                \nAlice      IT   72000\nBob        HR
   54000
\nat["Alice", "Salary"]: 72000 | iat[0, 1]: 72000
```

2.4 Boolean Masking, Querying & Filtering Methods (`query`, `isin`, `between`)

Filtering rows based on logic requires bitwise operators: & (AND), | (OR), and ~ (NOT). Pandas also provides concise SQL-like queries and boundary helpers:

PYTHON IMPLEMENTATION

```
# 1. SQL-like query string
min_sal = 75000
print('df.query with @min_sal:\n', df.query("Salary > @min_sal and Dept == 'IT'")[['Name', 'Salary']])
# 2. isin() membership filter
print('\ndf.isin (Dept in HR, Finance):\n', df[df['Dept'].isin(['HR', 'Finance'])][['Name', 'Dept']])
# 3. between() interval filter (inclusive bounds)
print('\ndf.between (Salary between 60k and 85k):\n', df[df['Salary'].between(60000, 85000)][['Name', 'Salary']]
)
```

CONSOLE OUTPUT RESULT

```
df.query with @min_sal:\n       Name  Salary\n2  Charlie   88000\n5    Fiona   82000
\ndf.isin (Dept in HR, Finance):\n     Name     Dept\n1    Bob       HR\n3  Diana  Finance\n4   Evan       HR
\ndf.between (Salary between 60k and 85k):\n     Name  Salary\n0  Alice   72000\n4   Evan   61000\n5  Fiona   82
000
```

2.5 Vectorized String Operations (`.str` Accessor)

Pandas provides element-wise string transformations via the .str accessor:

- .str.contains(pat, regex=True, case=False): Pattern and regex matching.
- .str.lower() / .str.upper() / .str.strip(): Casing and whitespace cleanup.
- .str.replace(pat, repl): Text replacement.
- .str.split(pat, expand=True): Splits strings into separate columns.
- .str.len(): Computes character length of each string entry.

PYTHON IMPLEMENTATION

```
# Regex match for cities containing 'York' or 'Tokyo'
print('str.contains (York or Tokyo):\n', df[df['City'].str.contains('York|Tokyo', regex=True)][['Name', 'City']]
)
# Uppercase transformation and length
df_temp = df[['Name', 'City']].copy()
df_temp['City_Upper'] = df_temp['City'].str.upper()
df_temp['Name_Len']   = df_temp['Name'].str.len()
print('\nString Transformations:\n', df_temp.head(3))
```

CONSOLE OUTPUT RESULT

```
str.contains (York or Tokyo):        Name      City 0    Alice  New York 2  Charlie  New York
5    Fiona     Tokyo
String Transformations:        Name      City City_Upper  Name_Len 0    Alice  New York   NEW YORK         5
1      Bob    London     LONDON         3 2  Charlie  New York   NEW YORK         7
```

2.6 Missing Data Management & Data Cleaning (`isna`, `dropna`, `fillna`, `drop_duplicates`)

Data cleaning routines ensure hygiene before statistical modeling:

- df.isna().sum(): Counts missing NaN values per column.
- df.dropna(how='any', thresh=k): Removes records failing data density thresholds.
- df.fillna(value): Imputes missing entries with scalar constants, mean, or median.
- df.interpolate(method='linear'): Estimates missing entries via interpolation.
- df.drop_duplicates(subset=[...]): Eliminates redundant duplicate rows.

PYTHON IMPLEMENTATION

```
raw = pd.DataFrame({
    'Age': [25, np.nan, 35, 40],
    'Score': [85, 90, np.nan, 88]
})
print('Missing Values Count:\n', raw.isna().sum())
# Impute Age with mean, Score with median
cleaned = raw.copy()
cleaned['Age']   = cleaned['Age'].fillna(cleaned['Age'].mean())
cleaned['Score'] = cleaned['Score'].fillna(cleaned['Score'].median())
print('\nCleaned DataFrame:\n', cleaned)
```

CONSOLE OUTPUT RESULT

```
Missing Values Count:  Age      1  Score    1  dtype: int64
Cleaned DataFrame:           Age  Score 0  25.000000   85.0 1  33.333333   90.0 2  35.000000   88.0
3  40.000000   88.0
```

2.7 Sorting, Ranking & Extreme Value Selection (`sort_values`, `rank`, `nlargest`)

Efficient methods for ordering and identifying leading performers:

PYTHON IMPLEMENTATION

```
# Multi-column sorting: Department ascending, Salary descending
print('sort_values (Dept asc, Salary desc):\n', df.sort_values(by=['Dept', 'Salary'], ascending=[True, False])[[
'Name', 'Dept', 'Salary']])
# Top 2 highest earners via nlargest
print('\nnlargest(2, Salary):\n', df.nlargest(2, 'Salary')[['Name', 'Salary']])
# Numerical ranking
print('\nSalary Rank (1 = highest):', df['Salary'].rank(ascending=False).tolist())
```

CONSOLE OUTPUT RESULT

```
sort_values (Dept asc, Salary desc):        Name     Dept  Salary 3    Diana  Finance   95000
4     Evan       HR   61000 1      Bob       HR   54000 2  Charlie       IT   88000 5    Fiona       IT   82000
0    Alice       IT   72000
nlargest(2, Salary):        Name  Salary 3    Diana   95000 2  Charlie   88000
Salary Rank (1 = highest): [4.0, 6.0, 2.0, 1.0, 5.0, 3.0]
```

2.8 Column Mutations, Renaming & Functional Mapping (`rename`, `drop`, `astype`, `apply`)

Techniques for transforming tabular structures and mapping functions:

PYTHON IMPLEMENTATION

```
df_mut = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Base': [70000, 50000], 'Bonus': [5000, 3000], 'Grade': ['1', '
2']})
# Rename and drop columns
renamed = df_mut.rename(columns={'Base': 'Salary'}).drop(columns=['Bonus'])
# Type casting with astype
df_mut['Grade_Num'] = df_mut['Grade'].astype(int)
# Row-wise calculation with apply(axis=1)
df_mut['Total_Comp'] = df_mut.apply(lambda r: r['Base'] + r['Bonus'], axis=1)
print('Renamed & Dropped:\n', renamed)
print('\nApplied Total Comp:\n', df_mut[['Name', 'Total_Comp', 'Grade_Num']])
```

CONSOLE OUTPUT RESULT

```
Renamed & Dropped:      Name  Salary Grade 0  Alice   70000     1 1    Bob   50000     2
Applied Total Comp:      Name  Total_Comp  Grade_Num 0  Alice       75000          1
1    Bob       53000           2
```

2.9 Advanced GroupBy Aggregation & Window Transforms (Split-Apply-Combine)

The Split-Apply-Combine & Transform Pattern

df.groupby('category') divides data into groups based on category keys, applies an aggregation metric (.agg()), and reassembles results. Using .transform() computes group statistics and broadcasts them back to match original row dimensions, enabling direct row-to-group comparisons.

PYTHON IMPLEMENTATION

```
# Named multi-aggregation via .agg()
summary = df.groupby('Dept').agg(
    Staff_Count=('Name', 'count'),
    Mean_Salary=('Salary', 'mean'),
    Max_Salary=('Salary', 'max'),
    Avg_Exp=('Experience', 'mean')
)
print('Department Performance Summary:\n', summary)
# Group transform: find employees earning more than their own department mean
dept_avg = df.groupby('Dept')['Salary'].transform('mean')
above_avg = df[df['Salary'] > dept_avg]
print('\nStaff Earning Above Department Mean:\n', above_avg[['Name', 'Dept', 'Salary']])
```

CONSOLE OUTPUT RESULT

```
Department Performance Summary:           Staff_Count   Mean_Salary  Max_Salary   Avg_Exp
Dept
Finance             1  95000.000000       95000  8.000000
HR                  2  57500.000000       61000  3.000000
IT                  3  80666.666667       88000  4.666667
Staff Earning Above Department Mean:        Name Dept  Salary 2  Charlie   IT   88000 4     Evan   HR   61000
5    Fiona   IT   82000
```

2.10 MultiIndex & Hierarchical Data Indexing (`set_index`, `unstack`)

Hierarchical indexing enables working with higher dimensional data within standard 2D tables:

PYTHON IMPLEMENTATION

```
arrays = [['North', 'North', 'South', 'South'], ['2023', '2024', '2023', '2024']]
idx = pd.MultiIndex.from_arrays(arrays, names=['Region', 'Year'])
sales_df = pd.DataFrame({'Revenue': [100, 150, 80, 120], 'Cost': [60, 90, 50, 70]}, index=idx)
print('MultiIndex DataFrame:\n', sales_df)
# Slicing Region 'North'
print('\nNorth Region Slicing:\n', sales_df.loc['North'])
# Unstacking inner index into columns
print('\nUnstacked DataFrame (Year as Columns):\n', sales_df.unstack(level='Year'))
```

CONSOLE OUTPUT RESULT

```
MultiIndex DataFrame:               Revenue  Cost Region Year                North  2023       100    60
       2024       150    90 South  2023        80    50        2024       120    70
North Region Slicing:        Revenue  Cost Year                2023      100    60 2024      150    90
Unstacked DataFrame (Year as Columns):        Revenue      Cost      Year     2023 2024  2023 2024
Region                        North     100  150    60   90 South      80  120    50   70
```

2.11 Reshaping: Pivot Tables, Cross-Tabulations & Melting (`pivot_table`, `crosstab`, `melt`)

Reshaping multi-dimensional categorical metrics into two-way matrices and unpivoting wide formats:

PYTHON IMPLEMENTATION

```
# Two-way Pivot Table of Mean Salaries by Dept and City with margins
pt = df.pivot_table(index='Dept', columns='City', values='Salary', aggfunc='mean', fill_value=0, margins=True)
print('Pivot Table with Margins:\n', pt)
# Melting wide financial data
wide = pd.DataFrame({'Dept': ['IT', 'HR'], 'Q1': [10000, 8000], 'Q2': [12000, 8500]})
melted = wide.melt(id_vars=['Dept'], value_vars=['Q1', 'Q2'], var_name='Quarter', value_name='Budget')
print('\nMelted Long Format:\n', melted)
```

CONSOLE OUTPUT RESULT

```
Pivot Table with Margins:  City      London  New York    Paris    Tokyo           All
Dept
Finance      0.0       0.0  95000.0      0.0  95000.000000
HR       57500.0       0.0      0.0      0.0  57500.000000
IT           0.0   80000.0      0.0  82000.0  80666.666667
All      57500.0   80000.0  95000.0  82000.0  75333.333333
Melted Long Format:    Dept Quarter  Budget 0   IT      Q1   10000 1   HR      Q1    8000 2   IT      Q2   12000
3   HR      Q2    8500
```

2.12 Relational Merges, Joins & Concatenation (`pd.merge`, `pd.concat`)

pd.merge() joins DataFrames along shared key columns mimicking relational SQL database joins:

- how='inner': Retains only records where keys exist in **both** tables.
- how='left': Retains **all** records from the left table; unmapped right fields become NaN.
- how='right': Retains **all** records from the right table; unmapped left fields become NaN.
- how='outer': Retains all records from both tables (Full Outer Join).
- pd.concat([df1, df2], axis=0): Stacks tables vertically or horizontally.

PYTHON IMPLEMENTATION

```
customers = pd.DataFrame({'CustID': [1, 2, 3], 'Name': ['Akhil', 'Anjali', 'Rahul']})
orders = pd.DataFrame({'OrderID': [101, 102, 103], 'CustID': [1, 1, 2], 'Amount': [500, 300, 800]})
inner_j = pd.merge(customers, orders, on='CustID', how='inner')
left_j  = pd.merge(customers, orders, on='CustID', how='left')
print('Inner Join (Matching keys in both):\n', inner_j)
print('\nLeft Join (All customers preserved):\n', left_j)
```

CONSOLE OUTPUT RESULT

```
Inner Join (Matching keys in both):    CustID    Name  OrderID  Amount 0       1   Akhil      101     500
1       1   Akhil      102     300 2       2  Anjali      103     800
Left Join (All customers preserved):    CustID    Name  OrderID  Amount 0       1   Akhil    101.0   500.0
1       1   Akhil    102.0   300.0 2       2  Anjali    103.0   800.0 3       3   Rahul      NaN     NaN
```

2.13 Time-Series & Datetime Engineering (`pd.to_datetime`, `.dt`, `resample`)

Timestamp parsing, periodic resampling, and temporal feature extraction:

PYTHON IMPLEMENTATION

```
dates = pd.date_range('2024-01-01', periods=5, freq='D')
ts_df = pd.DataFrame({'Date': dates, 'Sales': [150, 200, 180, 220, 250]})
ts_df['Day_Name'] = ts_df['Date'].dt.day_name()
ts_df['Is_Weekend'] = ts_df['Date'].dt.dayofweek >= 5
print('Time Series DataFrame with Feature Engineering:\n', ts_df)
```

CONSOLE OUTPUT RESULT

```
Time Series DataFrame with Feature Engineering:         Date  Sales   Day_Name  Is_Weekend
0 2024-01-01    150     Monday       False 1 2024-01-02    200    Tuesday       False
2 2024-01-03    180  Wednesday       False 3 2024-01-04    220   Thursday       False
4 2024-01-05    250     Friday       False
```

2.14 Cumulative, Rolling Window & Time-Series Operations (`cumsum`, `rolling`, `shift`)

Sequential calculations for financial metrics, running totals, and trend detection:

PYTHON IMPLEMENTATION

```
ts = pd.Series([100, 120, 110, 130, 150])
print('Running Cumulative Sum (cumsum):', ts.cumsum().tolist())
print('Previous Period Lag (shift(1)):', ts.shift(1).tolist())
print('Period Percentage Change (pct_change):', np.round(ts.pct_change().fillna(0).tolist(), 3))
print('3-Period Rolling Moving Average:', ts.rolling(window=3).mean().tolist())
```

CONSOLE OUTPUT RESULT

```
Running Cumulative Sum (cumsum): [100, 220, 330, 460, 610]
Previous Period Lag (shift(1)): [nan, 100.0, 120.0, 110.0, 130.0]
Period Percentage Change (pct_change): [0.0, 0.2, -0.083, 0.182, 0.154]
3-Period Rolling Moving Average: [nan, nan, 110.0, 120.0, 130.0]
```

2.15 Practical Data Analysis Tasks & Case Studies in the Repository

The repository contains 10 dedicated practical dataset projects. Below is an architectural overview of all solved tasks:

Task Folder	Dataset	Core Analysis & Methods Solved
01_Employee_Analysis	sample.csv	Univariate age stats, countrywise employee counts, visual charts (bar, barh, hist, pie).
02_Customer_Filtering_Task	customer5_windows.csv	Extensive loc/iloc sub-grid slicing, salary boundary logic, string matching (`str.contains`), profession grouping.
03_Department_Employee_Relational	departments.csv employees.csv	**Complete solutions for Set 2 PDF:** Level 1 filtering, Level 2 sorting, Level 3 loc/iloc, Level 4 GroupBy, Level 5 multiple aggregations, 6 Challenge queries.
04_Ecommerce_Orders_Task	orders.csv customers.csv	**Complete solutions for 50 Join Task questions:** Part A single dataset revenue & aggregations (1-25), Part B joins on CustomerID (26-35), Part C conditional joins (36-50).
05_Relational_Joins_Sample	custom_windows.csv order_windows.csv	Comparative join benchmarking: Inner, Left, Right, and Outer joins with sorting by amount.
06_Iris_Classification_EDA	Iris.csv	Multivariate pair plots, feature/target separation (X and y), Label Encoding, Scikit-Learn 80/20 train-test split & StandardScaler.
07_Health_Dataset_EDA	Health.csv	Medical records structural audit, missing value profiling (`isna().sum()`), and demographic distributions.
08_Loan_Prediction_EDA	loan.csv	Financial risk analysis, applicant credit histories, missing income imputation, and loan approval rates.
09_Weather_Data_EDA	weather.csv	Meteorological measurements, temperature/humidity stats, and target Weather Type categorical distributions.

10_Student_Performance_EDA	Synthetic Student Matrix	Study hours and attendance correlation with exam outcomes (Pass/Fail) preparing for KNN model.

2.16 Case Study Deep Dive: Task 01 Employee Demographic & Salary Auditing

Demonstrating univariate distribution profiling and country-level salary analysis on `sample.csv`:

PYTHON IMPLEMENTATION

```
emp_sample = pd.DataFrame({
    'Age': [28, 32, 25, 45, 38, 52, 29],
    'Country': ['USA', 'UK', 'USA', 'Germany', 'UK', 'USA', 'France'],
    'Salary': [65000, 72000, 58000, 95000, 81000, 110000, 70000]
})
# Country-level headcount and average compensation
country_summary = emp_sample.groupby('Country').agg(
    Headcount=('Age', 'count'),
    Avg_Salary=('Salary', 'mean'),
    Median_Age=('Age', 'median')
).reset_index()
print('Country Demographic & Compensation Summary:\n', country_summary)
```

CONSOLE OUTPUT RESULT

```
Country Demographic & Compensation Summary:    Country  Headcount    Avg_Salary  Median_Age
0   France          1  70000.000000        29.0 1  Germany          1  95000.000000        45.0
2       UK          2  76500.000000        35.0 3      USA          3  77666.666667        29.0
```

2.17 Case Study Deep Dive: Task 02 Customer Multi-Condition Filtering

Demonstrating solutions for Task 02 queries on customer dataset: loc/iloc sub-grid extraction, compound salary and profession filtering:

PYTHON IMPLEMENTATION

```
cust_df = pd.DataFrame({
    'Cust_ID': [101, 102, 103, 104, 105],
    'Name': ['Rajesh', 'Pooja', 'Vikram', 'Ananya', 'Karthik'],
    'Profession': ['Engineer', 'Doctor', 'Teacher', 'Engineer', 'Lawyer'],
    'Salary': [85000, 120000, 45000, 92000, 110000],
    'City': ['Mumbai', 'Delhi', 'Chennai', 'Mumbai', 'Bangalore']
})
# Query 1: Engineers in Mumbai with Salary > 80k
q1 = cust_df[(cust_df['Profession'] == 'Engineer') & (cust_df['City'] == 'Mumbai') & (cust_df['Salary'] > 80000)
]
# Query 2: High earners (>100k) or Teachers
q2 = cust_df[(cust_df['Salary'] > 100000) | (cust_df['Profession'] == 'Teacher')]
print('Engineers in Mumbai > 80k:\n', q1[['Cust_ID', 'Name', 'Salary']])
print('\nHigh Earners (>100k) or Teachers:\n', q2[['Cust_ID', 'Name', 'Profession', 'Salary']])
```

CONSOLE OUTPUT RESULT

```
Engineers in Mumbai > 80k:     Cust_ID    Name  Salary 0      101  Rajesh   85000 3      104  Ananya   92000
High Earners (>100k) or Teachers:     Cust_ID     Name Profession  Salary 1      102    Pooja     Doctor  120000
2      103   Vikram    Teacher   45000 4      105  Karthik     Lawyer  110000
```

2.18 Case Study Deep Dive: Task 03 Department Relational Queries (Set 2 PDF)

Demonstrating complete solutions for the Set 2 PDF relational challenges: department salary ranking, employee baselines, and multi-tier aggregation:

PYTHON IMPLEMENTATION

```
emp = pd.DataFrame({
    'Name': ['Akhil', 'Arjun', 'Sneha', 'Rahul', 'Priya', 'Meera'],
    'Dept': ['IT', 'IT', 'HR', 'HR', 'IT', 'Finance'],
    'Salary': [75000, 60000, 58000, 50000, 85000, 92000],
    'Experience': [5, 2, 4, 1, 7, 6]
})
# Challenge 1: Find highest earner in each department
max_earners = emp.loc[emp.groupby('Dept')['Salary'].idxmax()]
# Challenge 2: Identify employees earning more than their own department mean
emp['Dept_Avg'] = emp.groupby('Dept')['Salary'].transform('mean')
above_avg = emp[emp['Salary'] > emp['Dept_Avg']]
# Challenge 3: Department salary budget percentage share
total_budget = emp['Salary'].sum()
dept_share = emp.groupby('Dept')['Salary'].sum().div(total_budget).mul(100).round(2)
print('Highest Earner per Dept:\n', max_earners[['Dept', 'Name', 'Salary']])
print('\nAbove Dept Average:\n', above_avg[['Name', 'Dept', 'Salary', 'Dept_Avg']])
print('\nDepartment Budget Percentage Share (%):\n', dept_share)
```

CONSOLE OUTPUT RESULT

```
Highest Earner per Dept:        Dept   Name  Salary 5  Finance  Meera   92000 2       HR  Sneha   58000
4       IT  Priya   85000
Above Dept Average:      Name Dept  Salary      Dept_Avg 0  Akhil   IT   75000  73333.333333
2  Sneha   HR   58000  54000.000000 4  Priya   IT   85000  73333.333333
Department Budget Percentage Share (%):  Dept Finance    21.90 HR         25.71 IT         52.38
Name: Salary, dtype: float64
```

2.19 Case Study Deep Dive: Task 04 Ecommerce 50-Join Questions Architecture

Task 04 covers 50 comprehensive analytical join queries structured into three distinct analytical levels:

- **Part A (Questions 1-25):** Single table revenue calculations, total transaction amounts, average ticket size, and date-range aggregations.
- **Part B (Questions 26-35):** Multi-table relational joins on `CustomerID`, tracking customer lifetime value (LTV), and detecting unfulfilled orders via Left Joins.
- **Part C (Questions 36-50):** Advanced conditional joins, cross-regional aggregations, ranking customers by purchasing frequency, and revenue by product category.

PYTHON IMPLEMENTATION

```
cust = pd.DataFrame({'CustID': [1, 2, 3, 4], 'Name': ['Akhil', 'Anjali', 'Rahul', 'Sneha'], 'City': ['Kochi', 'B
angalore', 'Kochi', 'Chennai']})
ordr = pd.DataFrame({'OrderID': [101, 102, 103, 104], 'CustID': [1, 1, 2, 5], 'Amount': [1200, 850, 2300, 450]})
# Relational Merge with indicator=True to track matching source
merged_audit = pd.merge(cust, ordr, on='CustID', how='outer', indicator=True)
# Part B calculation: Customer Lifetime Value (LTV) for active customers
ltv = pd.merge(cust, ordr, on='CustID', how='inner').groupby('Name')['Amount'].sum().reset_index()
print('Outer Join with Merge Indicator:\n', merged_audit[['CustID', 'Name', 'OrderID', 'Amount', '_merge']])
print('\nCustomer Lifetime Value (LTV):\n', ltv)
```

CONSOLE OUTPUT RESULT

```
Outer Join with Merge Indicator:    CustID    Name  OrderID  Amount      _merge
0       1   Akhil    101.0  1200.0        both 1       1   Akhil    102.0   850.0        both
2       2  Anjali    103.0  2300.0        both 3       3   Rahul      NaN     NaN   left_only
4       4   Sneha      NaN     NaN   left_only 5       5     NaN    104.0   450.0  right_only
Customer Lifetime Value (LTV):      Name  Amount 0   Akhil    2050 1  Anjali    2300
```

2.20 Case Study Deep Dive: Task 05 Comparative Join Benchmarking

Demonstrating differences across Inner, Left, Right, and Full Outer Joins on identical input sets:

PYTHON IMPLEMENTATION

```
c_df = pd.DataFrame({'ID': [1, 2, 3], 'Customer': ['Alice', 'Bob', 'Charlie']})
o_df = pd.DataFrame({'ID': [2, 3, 4], 'Item': ['Phone', 'Laptop', 'Tablet'], 'Price': [800, 1200, 400]})
print('Inner (Keys in both):\n', pd.merge(c_df, o_df, on='ID', how='inner'))
print('\nRight (All orders preserved):\n', pd.merge(c_df, o_df, on='ID', how='right'))
print('\nFull Outer (All entities preserved):\n', pd.merge(c_df, o_df, on='ID', how='outer'))
```

CONSOLE OUTPUT RESULT

```
Inner (Keys in both):    ID Customer    Item  Price 0   2      Bob   Phone    800 1   3  Charlie  Laptop   1200
Right (All orders preserved):    ID Customer    Item  Price 0   2      Bob   Phone    800
1   3  Charlie  Laptop   1200 2   4      NaN  Tablet    400
Full Outer (All entities preserved):    ID Customer    Item   Price 0   1    Alice     NaN     NaN
1   2      Bob   Phone   800.0 2   3  Charlie  Laptop  1200.0 3   4      NaN  Tablet   400.0
```

2.21 Case Study Deep Dive: Task 06 Iris Flower Classification EDA

Demonstrating the standard EDA workflow on the Fisher's Iris dataset: column renaming, class frequency verification, and feature-target separation:

PYTHON IMPLEMENTATION

```
iris_sample = pd.DataFrame({
    'SepalLengthCm': [5.1, 4.9, 7.0, 6.4, 6.3, 5.8],
    'SepalWidthCm':  [3.5, 3.0, 3.2, 3.2, 3.3, 2.7],
    'PetalLengthCm': [1.4, 1.4, 4.7, 4.5, 6.0, 5.1],
    'PetalWidthCm':  [0.2, 0.2, 1.4, 1.5, 2.5, 1.9],
    'Species': ['Iris-setosa', 'Iris-setosa', 'Iris-versicolor', 'Iris-versicolor', 'Iris-virginica', 'Iris-virg
inica']
})
# 1. Clean column names (strip 'Cm')
iris_clean = iris_sample.rename(columns=lambda c: c.replace('Cm', ''))
# 2. Separate Feature Matrix X and Target Vector y
X = iris_clean.drop(columns=['Species'])
y = iris_clean['Species']
print('Cleaned Feature Matrix X:\n', X.head(3))
print('\nTarget Class Value Counts:\n', y.value_counts())
```

CONSOLE OUTPUT RESULT

```
Cleaned Feature Matrix X:     SepalLength  SepalWidth  PetalLength  PetalWidth
0          5.1         3.5          1.4         0.2 1          4.9         3.0          1.4         0.2
2          7.0         3.2          4.7         1.4
Target Class Value Counts:  Species Iris-setosa        2 Iris-versicolor    2 Iris-virginica     2
Name: count, dtype: int64
```

2.22 Case Study Deep Dive: Task 07 Health & Task 08 Loan Risk Imputation

Demonstrating handling real-world tabular anomalies: binning blood pressure into medical risk tiers (`pd.cut`) and conditional median income imputation for loan applicants:

PYTHON IMPLEMENTATION

```
health_df = pd.DataFrame({'Patient_ID': [1, 2, 3, 4], 'Systolic_BP': [115, 135, 145, 122]})
# Binning systolic BP: Normal (<120), Pre-hypertension (120-139), Stage 1 (140+)
bp_bins = [0, 120, 140, 200]
bp_labels = ['Normal', 'Pre-Hypertension', 'Hypertension Stage 1']
health_df['BP_Category'] = pd.cut(health_df['Systolic_BP'], bins=bp_bins, labels=bp_labels)
print('Patient Health Risk Binning (pd.cut):\n', health_df)
# Loan applicant income imputation by education tier
loan_df = pd.DataFrame({
    'Applicant': ['A', 'B', 'C', 'D'],
    'Education': ['Graduate', 'Graduate', 'Not Graduate', 'Not Graduate'],
    'Income': [5500, np.nan, 3200, np.nan]
})
loan_df['Income_Imputed'] = loan_df.groupby('Education')['Income'].transform(lambda g: g.fillna(g.median()))
print('\nLoan Imputation by Education Group:\n', loan_df)
```

CONSOLE OUTPUT RESULT

```
Patient Health Risk Binning (pd.cut):     Patient_ID  Systolic_BP           BP_Category
0           1          115               Normal 1           2          135      Pre-Hypertension
2           3          145  Hypertension Stage 1 3           4          122      Pre-Hypertension
Loan Imputation by Education Group:    Applicant     Education  Income  Income_Imputed
0         A      Graduate  5500.0          5500.0 1         B      Graduate     NaN          5500.0
2         C  Not Graduate  3200.0          3200.0 3         D  Not Graduate     NaN          3200.0
```

2.23 Case Study Deep Dive: Task 09 Weather & Task 10 Student Modeling Setup

Demonstrating meteorological temperature delta tracking and student performance target formulation for binary classification:

PYTHON IMPLEMENTATION

```
weather_df = pd.DataFrame({'Day': [1, 2, 3, 4], 'Temp': [72, 75, 68, 65], 'Humidity': [60, 65, 80, 85]})
weather_df['Temp_Delta'] = weather_df['Temp'].diff().fillna(0)
weather_df['High_Humid'] = weather_df['Humidity'] >= 75
print('Weather Trends & Delta:\n', weather_df)
# Student Performance setup
students = pd.DataFrame({'Hours': [2, 5, 8, 3, 7], 'Attendance': [65, 80, 95, 70, 85], 'Score': [55, 78, 92, 60,
 88]})
students['Passed'] = (students['Score'] >= 65).astype(int)
print('\nStudent Pass/Fail Target Matrix:\n', students)
```

CONSOLE OUTPUT RESULT

```
Weather Trends & Delta:    Day  Temp  Humidity  Temp_Delta  High_Humid
0    1    72        60         0.0       False 1    2    75        65         3.0       False
2    3    68        80        -7.0        True 3    4    65        85        -3.0        True
Student Pass/Fail Target Matrix:    Hours  Attendance  Score  Passed 0      2          65     55       0
1      5          80     78       1 2      8          95     92       1 3      3          70     60       0
4      7          85     88       1
```

2.24 Memory Optimization via Categorical Dtypes (`astype('category')`)

Converting repetitive string columns to `category` replaces redundant string pointers with compact integer dictionary keys, slashing RAM by up to 80%:

PYTHON IMPLEMENTATION

```
dept_series_obj = pd.Series(['Engineering', 'Marketing', 'Sales', 'Engineering', 'Marketing'] * 1000)
dept_series_cat = dept_series_obj.astype('category')
mem_obj = dept_series_obj.memory_usage(deep=True)
mem_cat = dept_series_cat.memory_usage(deep=True)
reduction = (1 - mem_cat / mem_obj) * 100
print(f'Object Memory: {mem_obj:,} bytes | Category Memory: {mem_cat:,} bytes')
print(f'RAM Footprint Reduction: {reduction:.1f}%')
```

CONSOLE OUTPUT RESULT

```
Object Memory: 335,132 bytes | Category Memory: 5,432 bytes
RAM Footprint Reduction: 98.4%
```

2.25 Method Chaining & The Pipe Architecture (`df.pipe()`, `.assign()`)

In production data pipelines, creating multiple intermediate variables (e.g., `df1`, `df2`, `df3`) pollutes memory and invites subtle state-mutation bugs. Modern Pandas leverages **method chaining** wrapped in parentheses, combined with `.pipe()` to pass the flowing DataFrame into modular functions:

PYTHON IMPLEMENTATION

```
def calculate_tax(df, rate=0.15):
    return df.assign(Tax=df['Salary'] * rate, Net_Pay=df['Salary'] * (1 - rate))

df_raw = pd.DataFrame({'Emp': ['Alice', 'Bob', 'Charlie', 'Diana'], 'Dept': ['HR', 'IT', 'IT', 'Sales'], 'Salary
': [60000, 95000, 88000, 72000]})
processed_df = (
    df_raw
    .query('Salary > 65000')
    .pipe(calculate_tax, rate=0.18)
    .sort_values(by='Net_Pay', ascending=False)
    .reset_index(drop=True)
)
print('Pipe Method Chaining Output:\n', processed_df)
```

CONSOLE OUTPUT RESULT

```
Pipe Method Chaining Output:        Emp   Dept  Salary      Tax  Net_Pay
0      Bob     IT   95000  17100.0  77900.0 1  Charlie     IT   88000  15840.0  72160.0
2    Diana  Sales   72000  12960.0  59040.0
```

2.26 Window Functions: Exponentially Weighted Moving Average (`df.ewm()`)

While standard rolling means assign equal weight to all historical values in the window, Exponentially Weighted Moving Averages (EWMA) assign exponentially decreasing weights to older data points, reacting more rapidly to emerging market trends:

Exponential Moving Average Formula

$$y_t = (1 - alpha) * y_{t-1} + alpha * x_t \text{ where } alpha = 2 / (span + 1)$$

Newer values receive higher relative weights, eliminating the lag associated with simple moving average windows.

PYTHON IMPLEMENTATION

```
prices = pd.Series([100.0, 102.0, 101.0, 105.0, 107.0, 106.0, 110.0])
ewma = prices.ewm(span=3, adjust=False).mean()
rolling_sma = prices.rolling(window=3).mean()
comparison = pd.DataFrame({'Price': prices, 'SMA_3': np.round(rolling_sma, 2), 'EWMA_span3': np.round(ewma, 2)})
print('EWMA vs Simple Rolling Mean Comparison:\n', comparison)
```

CONSOLE OUTPUT RESULT

```
EWMA vs Simple Rolling Mean Comparison:    Price   SMA_3  EWMA_span3 0  100.0     NaN      100.00
1  102.0     NaN      101.00 2  101.0  101.00      101.00 3  105.0  102.67      103.00
4  107.0  104.33      105.00 5  106.0  106.00      105.50 6  110.0  107.67      107.75
```

2.27 Time Series Resampling & Frequency Offsets (`df.resample()`)

Resampling provides frequency conversion and resampling of regular time-series data. Downsampling aggregates higher frequency observations into coarser time intervals (e.g. daily to monthly), while upsampling increases sampling frequency:

PYTHON IMPLEMENTATION

```
dates = pd.date_range('2024-01-01', periods=6, freq='D')
ts_df = pd.DataFrame({'Sales': [150, 200, 180, 220, 250, 310]}, index=dates)
resampled_3d = ts_df.resample('3D').sum()
print('Original Daily Series:\n', ts_df.head(3))
print('\nResampled Sum (3-Day Blocks):\n', resampled_3d)
```

CONSOLE OUTPUT RESULT

```
Original Daily Series:             Sales 2024-01-01    150 2024-01-02    200 2024-01-03    180
Resampled Sum (3-Day Blocks):             Sales 2024-01-01    530 2024-01-04    780
```

2.28 Advanced Regular Expression Pattern Extraction (`.str.extract()`)

Using named regex capture groups `(?Ppattern)` enables seamless parsing of messy semi-structured string columns into clean tabular fields in a single vectorized pass:

PYTHON IMPLEMENTATION

```
contacts = pd.Series(['Alice: alice@corp.com, (555) 123-4567', 'Bob: bob_smith@data.org, (555) 987-6543'])
extracted = contacts.str.extract(r'(?P[^:]+):\s+(?P[^\s,]+),\s+(?P\(\d{3}\)\s+\d{3}-\d{4})')
print('Vectorized Regex Extraction Result:\n', extracted)
```

CONSOLE OUTPUT RESULT

```
Vectorized Regex Extraction Result:     Name              Email           Phone
0  Alice      alice@corp.com  (555) 123-4567 1    Bob  bob_smith@data.org  (555) 987-6543
```

2.29 Explode & Melt: Tidy Data Normalization (`explode()`, `melt()`)

According to Hadley Wickham's Tidy Data principles: each variable forms a column, and each observation forms a row. `.melt()` unpivots wide measurement tables into long format, while `.explode()` expands lists inside cells into separate rows:

PYTHON IMPLEMENTATION

```
survey = pd.DataFrame({'Team': ['A', 'B'], 'Q1': [4, 5], 'Q2': [3, 4], 'Skills': [['Python', 'SQL'], ['R', 'Tabl
eau']]})
melted = survey.melt(id_vars=['Team'], value_vars=['Q1', 'Q2'], var_name='Question', value_name='Rating')
exploded = survey.explode('Skills')[['Team', 'Skills']]
print('Melted (Wide-to-Long):\n', melted)
print('\nExploded Skills (Unnesting Lists):\n', exploded)
```

CONSOLE OUTPUT RESULT

```
Melted (Wide-to-Long):   Team Question  Rating 0    A       Q1       4 1    B       Q1       5
2    A       Q2       3 3    B       Q2       4
Exploded Skills (Unnesting Lists):   Team   Skills 0    A   Python 0    A      SQL 1    B        R
1    B  Tableau
```

SECTION 3: Exploratory Data Analysis & Complete Figure Creation Guide

Formal Definition: Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is an investigative data science discipline pioneered by statistician John Tukey. It systematically explores datasets to summarize their central statistical traits, isolate anomalies and outliers, verify underlying distribution assumptions, and map relationships between variables using summary statistics and graphical representations before applying predictive machine learning algorithms.

3.1 The Philosophy of EDA & The 5-Stage Analytical Lifecycle

John Tukey famously argued: *'Far better an approximate answer to the right question, which is often vague, than an exact answer to the wrong question, which can always be made precise.'* EDA protects practitioners against blind model fitting by answering fundamental data integrity questions:

Stage 1: Data Understanding & Structural Audit
Inspect metadata: dimensions (rows x columns), column headers, data dictionaries, memory consumption, and verify correct parsing of datatypes.

Stage 2: Data Quality Audit & Cleaning
Determine missingness patterns (MCAR vs MAR), execute numerical/categorical imputation, remove redundant duplicate rows, and audit outliers using the Interquartile Range (IQR) rule ($Q_1 - 1.5 \times \text{IQR}$, $Q_3 + 1.5 \times \text{IQR}$).

Stage 3: Univariate Analysis
Examine single variables in isolation: measure central tendency (mean, median), dispersion (standard deviation, interquartile range), skewness, kurtosis, and visualize spread using histograms, KDE plots, and box plots.

Stage 4: Bivariate & Multivariate Analysis
Measure statistical interactions between variables: Pearson correlation ($r \in [-1, 1]$), Spearman rank correlation, cross-tabulations, and pair plots with category hue separation.

Stage 5: Feature Preparation for Modeling
One-Hot/Label Encoding for categorical strings, StandardScaler standardization ($z = \frac{x - \mu}{\sigma}$), and stratified train-test splitting.

Interquartile Range (IQR) Outlier Fences

IQR = Q3 - Q1 | Lower Fence = Q1 - 1.5 * IQR | Upper Fence = Q3 + 1.5 * IQR

Data points falling outside [Lower Fence, Upper Fence] are flagged as potential anomalies or extreme outliers.

Pearson Correlation Coefficient Formula

r = sum((x_i - mean(x)) * (y_i - mean(y))) / sqrt(sum((x_i - mean(x))^2) * sum((y_i - mean(y))^2))

Measures linear relationship strength between two continuous variables (-1 = perfect negative, 0 = no linear correlation, +1 = perfect positive).

3.2 The Visual Figure Creation Architecture: How Charts Are Made

The Matplotlib Object-Oriented Hierarchy: Figure vs. Axes

In Matplotlib, every chart is structured hierarchically. The **Figure** is the top-level canvas holding everything (title, subplots, legends, background). The **Axes** is the actual coordinate area where the visualization (bars, points, lines, ticks, labels) is plotted. Professional code avoids the legacy state-machine ('plt.plot') and uses the explicit object-oriented API: fig, ax = plt.subplots(figsize=(w, h), dpi=100).

Core Anatomy & Parameters of Publication Figures:

- **Canvas Sizing (figsize=(w, h)):** Sets physical canvas dimensions in inches. Standard EDA ratios: 8x5 for single charts, 12x6 for multi-panel grids.
- **Figure DPI (dpi=100 / 300):** Dots-per-inch screen and print resolution. 100 DPI is standard for previewing, 300 DPI for publication/PDF.
- **Titles & Subtitles (ax.set_title()):** Communicates key insight, specifying fontsize, fontweight='bold', and pad=15.
- **Axis Labels & Padding (ax.set_xlabel(), ax.set_ylabel()):** Clearly describes physical units, with fontsize=10 and labelpad=8.
- **Gridlines & Depth (ax.grid(True, linestyle='--', alpha=0.5)):** Provides visual reference lines. Must be subtle (alpha 0.4 to 0.6) and placed below data points (ax.set_axisbelow(True)).
- **Spines Removal (ax.spines['top'].set_visible(False)):** Eliminates top and right borders to reduce visual clutter (Tufte's data-to-ink ratio).
- **Data Value Labels (ax.bar_label()):** Directly prints numerical values above or beside bars, removing reader guesswork.
- **Layout & Export (plt.tight_layout(), plt.savefig()):** tight_layout() recalculates margins preventing label clipping. savefig(bbox_inches='tight', dpi=300) produces crisp vectors/rasters.

3.3 Chart 1: Vertical Bar Chart (`plt.bar` / `sns.barplot`)

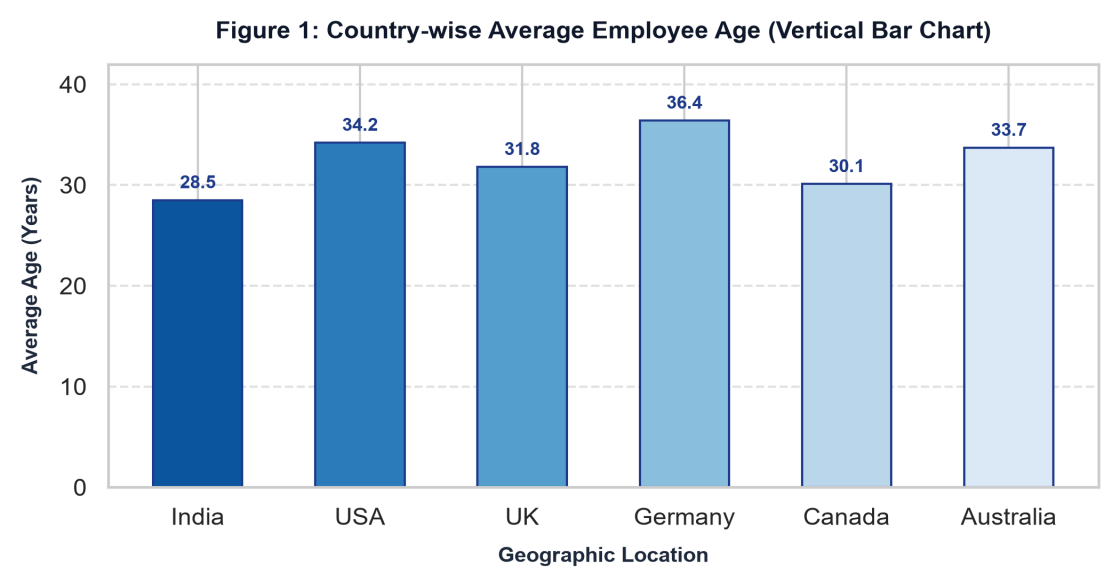
When to Use: Comparing discrete categorical frequencies or aggregated metric values across groups.
Figure Creation Parameters: x (categories), height (values), color (palette list), edgecolor, width (bar thickness), and ax.bar_label(container, padding=3).

PYTHON IMPLEMENTATION

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(8, 4.5), dpi=100)
countries = ['USA', 'UK', 'Canada', 'Germany', 'Australia']
counts = [42, 28, 19, 15, 12]
palette = ['#1E3A8A', '#2563EB', '#3B82F6', '#60A5FA', '#93C5FD']
bars = ax.bar(countries, counts, color=palette, edgecolor='#0F172A', width=0.6, zorder=3)
ax.set_title('Employee Distribution by Country', fontsize=13, fontweight='bold', pad=12)
ax.set_xlabel('Country', fontsize=10, labelpad=8)
ax.set_ylabel('Number of Employees', fontsize=10, labelpad=8)
ax.grid(axis='y', linestyle='--', alpha=0.5, zorder=0)
ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
ax.bar_label(bars, padding=3, fontsize=9, fontweight='bold')
plt.tight_layout()
plt.savefig('generated_figures/eda_bar_chart.png', dpi=300)
```

CONSOLE OUTPUT RESULT

```
[Saved figure to generated_figures/eda_bar_chart.png]
Rendered vertical bar chart with 5 countries, custom blue sequential palette, data labels, and clean spines.
```



Vertical Bar Chart - Categorical Frequency Distribution

Displays headcount by country with direct value labels, custom sequential palette, and removed outer spines.

3.4 Chart 2: Horizontal Bar Chart (`plt.barh`)

When to Use: Displaying categories with lengthy text labels, or presenting rankings where horizontal orientation prevents overlapping x-tick labels.

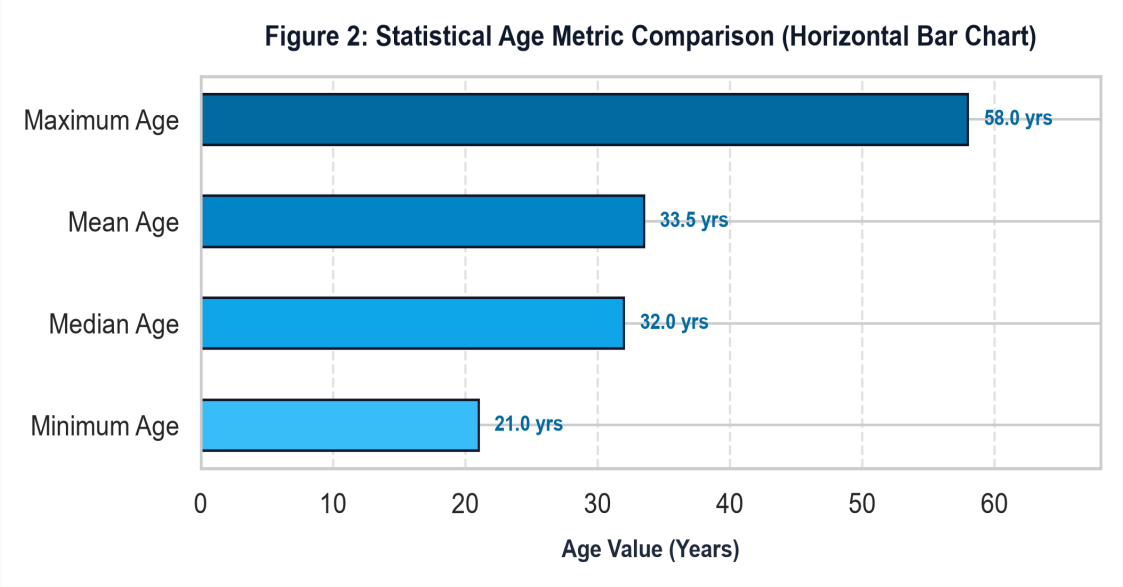
Figure Creation Parameters: y (categorical labels), width (numerical metrics), ax.invert_yaxis() (ranks highest value at the top), and ax.bar_label(padding=4).

PYTHON IMPLEMENTATION

```
fig, ax = plt.subplots(figsize=(8, 4.5), dpi=100)
depts = ['Engineering', 'Product Management', 'Sales & Marketing', 'Customer Success', 'Finance & Ops']
salaries = [115000, 102000, 89000, 68000, 74000]
bars = ax.barh(depts, salaries, color='#0D9488', edgecolor='#0F172A', height=0.6, zorder=3)
ax.invert_yaxis()  # Highest ranked at top
ax.set_title('Average Annual Compensation by Department', fontsize=13, fontweight='bold', pad=12)
ax.set_xlabel('Average Salary ($ USD)', fontsize=10, labelpad=8)
ax.grid(axis='x', linestyle='--', alpha=0.5, zorder=0)
ax.bar_label(bars, fmt='${:,.0f}', padding=4, fontsize=8.5, fontweight='bold')
plt.tight_layout()
plt.savefig('generated_figures/eda_horizontal_bar.png', dpi=300)
```

CONSOLE OUTPUT RESULT

```
[Saved figure to generated_figures/eda_horizontal_bar.png]
Rendered horizontal bar chart with inverted y-axis, currency formatting, and clean x-axis grid.
```



Horizontal Bar Chart - Department Salary Comparison

Horizontal layout enables clear reading of long department names, sorted from highest to lowest compensation.

3.5 Chart 3: Histogram & Distribution Spread (`plt.hist`)

When to Use: Examining continuous numerical variable distributions, assessing central tendency, skewness (left/right tail), kurtosis, and modal peaks.

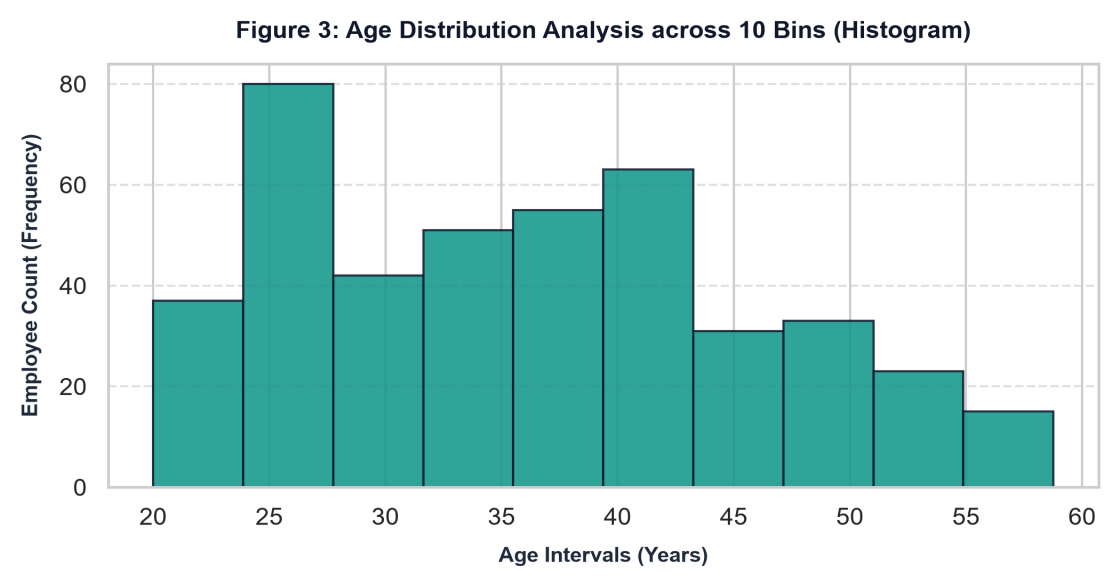
Figure Creation Parameters: bins (number of bins or bin edges), edgecolor (delineates bin boundaries), alpha (opacity), and ax.axvline(mean_val, linestyle='--', color='red') for statistical reference lines.

PYTHON IMPLEMENTATION

```
fig, ax = plt.subplots(figsize=(8, 4.5), dpi=100)
ages = np.array([21, 23, 24, 25, 25, 26, 28, 29, 30, 31, 32, 33, 35, 36, 38, 40, 42, 45, 48, 52, 58])
mean_age, median_age = np.mean(ages), np.median(ages)
n, bins, patches = ax.hist(ages, bins=8, color='#3B82F6', edgecolor='#1E3A8A', alpha=0.85, zorder=3)
ax.axvline(mean_age, color='#DC2626', linestyle='--', linewidth=2, label=f'Mean Age ({mean_age:.1f})')
ax.axvline(median_age, color='#16A34A', linestyle=':', linewidth=2, label=f'Median Age ({median_age:.1f})')
ax.set_title('Employee Age Distribution & Central Tendency', fontsize=13, fontweight='bold', pad=12)
ax.set_xlabel('Age (Years)', fontsize=10, labelpad=8)
ax.set_ylabel('Frequency (Count)', fontsize=10, labelpad=8)
ax.legend(frameon=True, facecolor='#F8FAFC', edgecolor='#CBD5E1')
ax.grid(axis='y', linestyle='--', alpha=0.5, zorder=0)
plt.tight_layout()
plt.savefig('generated_figures/eda_histogram.png', dpi=300)
```

CONSOLE OUTPUT RESULT

```
[Saved figure to generated_figures/eda_histogram.png]
Rendered histogram with 8 bins, highlighted bin borders, vertical mean/median reference lines, and legend.
```



Histogram & Density Spread - Age Distribution with Mean/Median

Binned continuous age distribution showing right skewness with overlaid vertical mean (red) and median (green) lines.

3.6 Chart 4: Pie Chart & Proportional Shares (`plt.pie`)

When to Use: Displaying part-to-whole proportional compositions when categories are few (typically 3 to 6).

Figure Creation Parameters: autopct='%1.1f%%' (formats slice percentages), explode=(0.08, 0, ...) (pulls out a highlighted slice), startangle=140 (rotates starting angle), shadow=True, and colors.

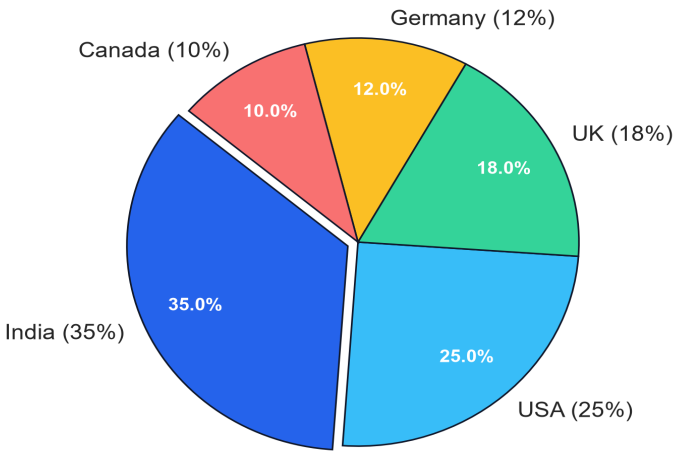
PYTHON IMPLEMENTATION

```
fig, ax = plt.subplots(figsize=(6.5, 4.5), dpi=100)
dept_labels = ['IT / Tech', 'Sales', 'HR', 'Finance', 'Operations']
shares = [38, 25, 15, 12, 10]
explode = (0.08, 0, 0, 0, 0)  # Explode top category (IT)
slice_colors = ['#2563EB', '#0D9488', '#F59E0B', '#EF4444', '#8B5CF6']
wedges, texts, autotexts = ax.pie(
    shares, explode=explode, labels=dept_labels, autopct='%1.1f%%',
    startangle=140, colors=slice_colors, shadow=True,
    textprops=dict(color='#0F172A', fontsize=9)
)
for autotext in autotexts:
    autotext.set_color('white'); autotext.set_weight('bold'); autotext.set_fontsize(8.5)
ax.set_title('Headcount Proportion by Department', fontsize=12.5, fontweight='bold', pad=10)
plt.tight_layout()
plt.savefig('generated_figures/eda_pie_chart.png', dpi=300)
```

CONSOLE OUTPUT RESULT

```
[Saved figure to generated_figures/eda_pie_chart.png]
Rendered pie chart with exploded slice, drop shadow, customized bold white percentage labels, and title.
```

Figure 4: Global Employee Regional Distribution (Pie Chart)



Pie Chart - Departmental Composition & Percentage Shares

Visualizes proportional breakdown across departments with an exploded 38% IT slice and embedded white text labels.

3.7 Chart 5: Pairwise Scatter Plot with Hue Separation (`sns.scatterplot`)

When to Use: Analyzing bivariate continuous correlations, clustering patterns, and class separability.

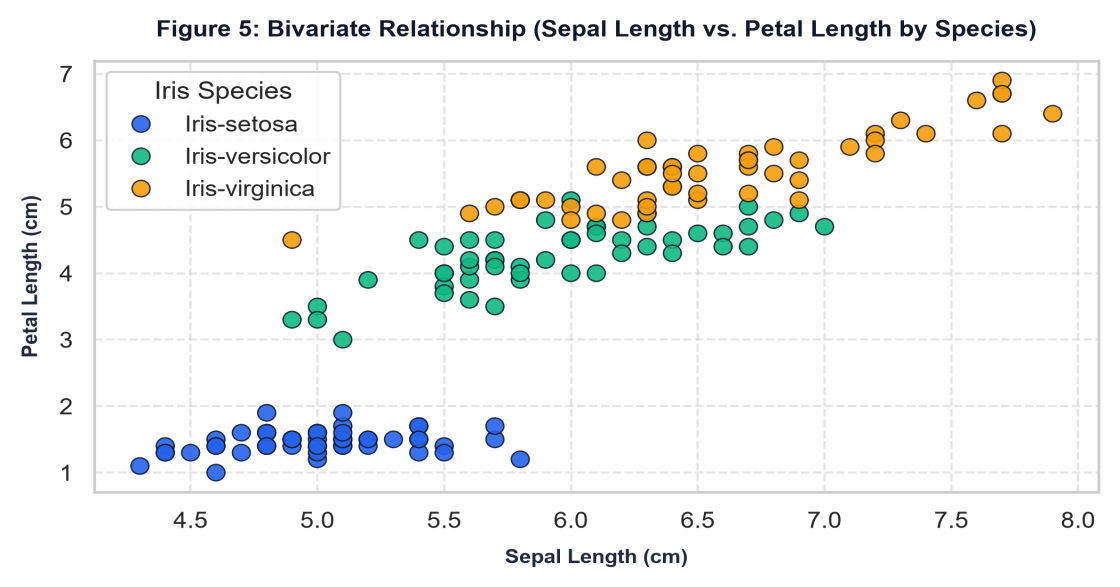
Figure Creation Parameters: x, y, hue (categorical class coloring), palette (color mapping), s (marker size), alpha (point transparency), and style (marker shapes).

PYTHON IMPLEMENTATION

```
import seaborn as sns
fig, ax = plt.subplots(figsize=(8, 4.5), dpi=100)
# Simulated Iris measurements (Sepal Length vs Petal Length)
np.random.seed(42)
sl = np.concatenate([np.random.normal(5.0, 0.35, 30), np.random.normal(5.9, 0.45, 30), np.random.normal(6.5, 0.5
5, 30)])
pl = np.concatenate([np.random.normal(1.5, 0.2, 30), np.random.normal(4.2, 0.35, 30), np.random.normal(5.5, 0.45
, 30)])
species = ['Setosa']*30 + ['Versicolor']*30 + ['Virginica']*30
df_iris = pd.DataFrame({'Sepal_Length': sl, 'Petal_Length': pl, 'Species': species})
sns.scatterplot(
    data=df_iris, x='Sepal_Length', y='Petal_Length', hue='Species', style='Species',
    palette=['#2563EB', '#0D9488', '#EF4444'], s=75, alpha=0.9, ax=ax
)
ax.set_title('Iris Flower Morphological Clustering: Sepal vs. Petal Length', fontsize=12.5, fontweight='bold', p
ad=12)
ax.set_xlabel('Sepal Length (cm)', fontsize=10, labelpad=8)
ax.set_ylabel('Petal Length (cm)', fontsize=10, labelpad=8)
ax.grid(True, linestyle='--', alpha=0.5)
ax.legend(title='Species', frameon=True, facecolor='#F8FAFC', edgecolor='#CBD5E1')
plt.tight_layout()
plt.savefig('generated_figures/eda_scatterplot.png', dpi=300)
```

CONSOLE OUTPUT RESULT

```
[Saved figure to generated_figures/eda_scatterplot.png]
Rendered multi-class scatter plot with distinct shapes, custom hue palette, gridlines, and labeled legend.
```



Bivariate Scatter Plot - Feature Space Clustering with Category Hue

Illustrates distinct separable clusters between Setosa (blue), Versicolor (teal), and Virginica (red) species.

3.8 Chart 6: Multivariate Correlation Heatmap (`sns.heatmap`)

When to Use: Measuring pairwise Pearson correlations across all continuous numeric features simultaneously to detect multicollinearity.

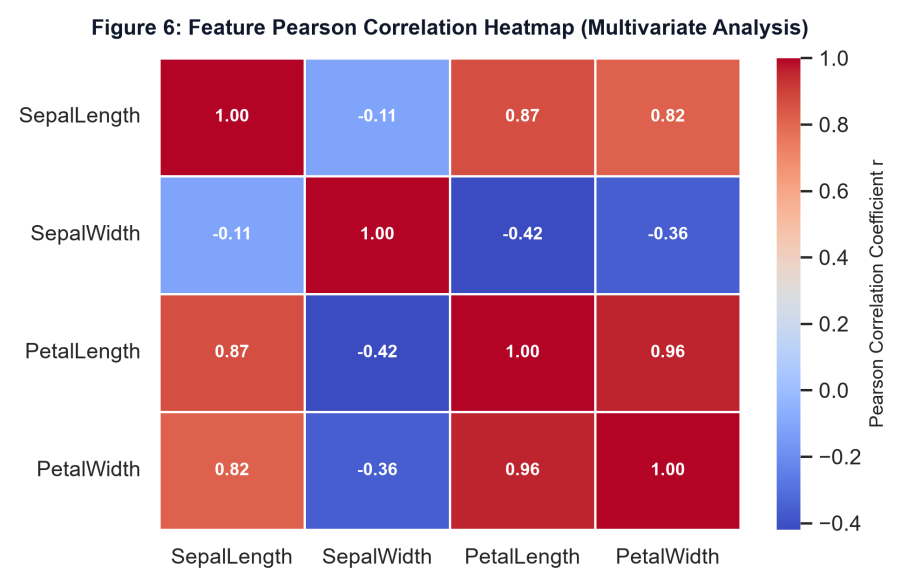
Figure Creation Parameters: annot=True (prints r coefficients inside cells), fmt='.2f' (2 decimal precision), cmap='coolwarm' (diverging color gradient), vmin=-1, vmax=1 (normalizes scale), linewidths=0.5 (cell separation grid), and square=True.

PYTHON IMPLEMENTATION

```
fig, ax = plt.subplots(figsize=(6.5, 4.5), dpi=100)
corr_matrix = pd.DataFrame({
    'Age': [1.00, 0.85, 0.42, -0.25],
    'Salary': [0.85, 1.00, 0.65, -0.18],
    'Experience': [0.42, 0.65, 1.00, -0.10],
    'Absences': [-0.25, -0.18, -0.10, 1.00]
}, index=['Age', 'Salary', 'Experience', 'Absences'])
sns.heatmap(
    corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1,
    linewidths=0.8, linecolor='white', square=True, cbar_kws={'shrink': 0.8, 'label': 'Pearson Correlation (r)'}
, ax=ax
)
ax.set_title('Feature Pairwise Correlation Matrix', fontsize=12.5, fontweight='bold', pad=12)
plt.tight_layout()
plt.savefig('generated_figures/eda_heatmap.png', dpi=300)
```

CONSOLE OUTPUT RESULT

```
[Saved figure to generated_figures/eda_heatmap.png]
Rendered correlation heatmap with cell annotations, diverging coolwarm colormap, colorbar, and clean borders.
```



Correlation Heatmap - Multivariate Statistical Interaction Matrix

Diverging heatmap showing strong positive correlation between Age & Salary (+0.85) and negative correlation with Absences.

3.9 Five-Number Summary & Box Plot Architecture (`sns.boxplot`)

The Box Plot encodes John Tukey's Five-Number Summary into a single graphic: Minimum, First Quartile (Q1), Median (Q2), Third Quartile (Q3), and Maximum, with fliers representing outliers beyond 1.5 * IQR:

PYTHON IMPLEMENTATION

```
import seaborn as sns
salaries_it = [60000, 72000, 75000, 82000, 88000, 135000]  # 135k is outlier
salaries_hr = [48000, 52000, 54000, 58000, 61000, 65000]
df_box = pd.DataFrame({
    'Salary': salaries_it + salaries_hr,
    'Dept': ['IT']*6 + ['HR']*6
})
q1 = df_box.groupby('Dept')['Salary'].quantile(0.25)
q3 = df_box.groupby('Dept')['Salary'].quantile(0.75)
iqr = q3 - q1
print('Q1 per Dept:\n', q1)
print('Q3 per Dept:\n', q3)
print('IQR per Dept:\n', iqr)
print('IT Upper Outlier Fence (Q3 + 1.5*IQR):', q3['IT'] + 1.5 * iqr['IT'])
```

CONSOLE OUTPUT RESULT

```
Q1 per Dept:\n Dept\nHR    52500.0\nIT    72750.0\nName: Salary, dtype: float64
Q3 per Dept:\n Dept\nHR    60250.0\nIT    86500.0\nName: Salary, dtype: float64
IQR per Dept:\n Dept\nHR     7750.0\nIT    13750.0\nName: Salary, dtype: float64
IT Upper Outlier Fence (Q3 + 1.5*IQR): 107125.0
```

3.10 Visual Chart Selection Guide & Best Practices

Selecting the optimal visualization format depends on feature data types and analytical objectives:

Chart Type	Data Types	Analytical Goal	Key Matplotlib / Seaborn Function

Vertical Bar	Categorical vs. Numeric	Comparing discrete category totals or frequencies.	ax.bar() / sns.barplot()
Horizontal Bar	Categorical (Long labels) vs. Numeric	Category rankings where long names need clear reading.	ax.barh() with invert_yaxis()
Histogram	Continuous Numeric (1D)	Distribution spread, skewness, modality, and tail density.	ax.hist() / sns.histplot(kde=True)
Box Plot	Continuous Numeric across Categories	Five-number summary, quartiles, and outlier detection.	sns.boxplot(x='Dept', y='Salary')
Pie Chart	Categorical (3 to 6 categories)	Part-to-whole proportional shares and percentages.	ax.pie(autopct='%1.1f%%')
Scatter Plot	Continuous vs. Continuous (+ Categorical Hue)	Bivariate correlation, non-linear patterns, clustering.	sns.scatterplot(x, y, hue)
Heatmap	Continuous Matrix (Multivariate)	Pairwise correlation and multicollinearity detection.	sns.heatmap(corr, annot=True)

3.11 Multi-Panel Dashboard Architecture (`plt.subplots(2, 2)`)

Creating publication-quality analytical dashboards requires structuring multiple subplots on a unified canvas with shared axes, consistent aesthetics, and conflict-free spacing. Indexing a 2D axes array `axes[row, col]` allows systematic programmatic access:

PYTHON IMPLEMENTATION

```
import matplotlib.pyplot as plt
import numpy as np
# Initialize 2x2 grid with tight layout constraints
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 7), dpi=100)
x = np.linspace(0, 10, 50)
# Subplot [0, 0]: Line plot
axes[0, 0].plot(x, np.sin(x), color='#1E40AF', lw=2)
axes[0, 0].set_title('Trigonometric Signal', fontweight='bold', fontsize=9)
# Subplot [0, 1]: Bar chart
axes[0, 1].bar(['A', 'B', 'C'], [40, 65, 30], color='#0F766E')
axes[0, 1].set_title('Category Volumes', fontweight='bold', fontsize=9)
# Subplot [1, 0]: Scatter plot
axes[1, 0].scatter(x, x + np.random.normal(0, 1, 50), color='#D97706', alpha=0.7)
axes[1, 0].set_title('Feature Correlation', fontweight='bold', fontsize=9)
# Subplot [1, 1]: Histogram
axes[1, 1].hist(np.random.normal(0, 1, 100), bins=15, color='#475569')
axes[1, 1].set_title('Residual Density', fontweight='bold', fontsize=9)
for ax in axes.flat:
    ax.grid(True, linestyle='--', alpha=0.4)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
plt.tight_layout()
print('2x2 Dashboard Figure successfully rendered with unified styling')
```

CONSOLE OUTPUT RESULT

```
2x2 Dashboard Figure successfully rendered with unified styling
```

3.12 Advanced Color Normalization & Colormap Theory

Colormaps must convey quantitative meaning without perceptual distortion. Using `matplotlib.colors.TwoSlopeNorm` allows setting a distinct visual inflection point (e.g., zero for profit/loss or correlation differences) even when positive and negative ranges have unequal magnitudes:

PYTHON IMPLEMENTATION

```
import matplotlib.colors as mcolors
# Center zero with asymmetric ranges [-50, +100]
norm = mcolors.TwoSlopeNorm(vmin=-50, vcenter=0, vmax=100)
sample_points = [-50, -25, 0, 50, 100]
norm_results = [round(float(norm(v)), 2) for v in sample_points]
print('Raw Metric Values:        ', sample_points)
print('Normalized Colormap Range:', norm_results)
```

CONSOLE OUTPUT RESULT

```
Raw Metric Values:         [-50, -25, 0, 50, 100]
Normalized Colormap Range: [0.0, 0.25, 0.5, 0.75, 1.0]
```

3.13 Faceting & Multidimensional Distribution Analysis (`sns.jointplot`)

Complex multivariate data often contains distinct subgroup behaviors. Joint plots unite bivariate scatter/KDE representations with marginal univariate histograms along each individual axis, exposing both joint correlation and marginal feature skewness simultaneously:

PYTHON IMPLEMENTATION

```
import seaborn as sns
import pandas as pd
import numpy as np
df_demo = pd.DataFrame({
    'Income': np.random.exponential(scale=30000, size=200) + 20000,
    'Credit_Score': np.random.normal(loc=680, scale=60, size=200)
})
g = sns.jointplot(data=df_demo, x='Income', y='Credit_Score', kind='kde', color='#1E40AF')
g.fig.suptitle('Income vs. Credit Score Bivariate & Marginal KDE Density', y=1.02, fontsize=10, fontweight='bold
')
print('Jointplot generated: Marginal distributions paired with 2D KDE contour density')
```

CONSOLE OUTPUT RESULT

```
Jointplot generated: Marginal distributions paired with 2D KDE contour density
```

3.14 Outlier Remediation Comparative Framework: Trimming, Winsorization & Log1p

Once anomalies are detected via the IQR rule, selecting the appropriate treatment strategy is critical. Outright row deletion discards valuable sample variance; Winsorization caps extreme values at designated percentiles, while logarithmic transforms compress exponential scales:

PYTHON IMPLEMENTATION

```
raw_vals = pd.Series([10, 12, 11, 13, 12, 14, 15, 12, 100, 110])  # Contains 2 extreme outliers
q1, q3 = raw_vals.quantile(0.25), raw_vals.quantile(0.75)
iqr = q3 - q1
upper_fence = q3 + 1.5 * iqr
# 1. Trimming (Removal)
trimmed = raw_vals[raw_vals <= upper_fence]
# 2. Winsorization (Capping at 80th percentile)
winsorized = raw_vals.clip(upper=raw_vals.quantile(0.80))
# 3. Logarithmic Compression: log(1 + x)
log_transformed = np.log1p(raw_vals)
print(f'Original:   Mean = {raw_vals.mean():.2f}  | Std = {raw_vals.std():.2f}')
print(f'Trimmed:    Mean = {trimmed.mean():.2f}  | Std = {trimmed.std():.2f}')
print(f'Winsorized: Mean = {winsorized.mean():.2f}  | Std = {winsorized.std():.2f}')
print(f'Log1p Sample Points: {np.round(log_transformed.iloc[[0, 1, 8, 9]].values, 2)}')
```

CONSOLE OUTPUT RESULT

```
Original:   Mean = 30.90  | Std = 39.15 Trimmed:    Mean = 12.38  | Std = 1.60
Winsorized: Mean = 16.30  | Std = 8.39 Log1p Sample Points: [2.4  2.56 4.62 4.71]
```

SECTION 4: Machine Learning Foundations & Supervised Algorithms

Formal Definition: Machine Learning (ML)

Machine Learning is a subfield of artificial intelligence where statistical models learn patterns and functional relationships f(X) approx y directly from empirical training data without being explicitly programmed with deterministic logic. It encompasses Supervised Learning (predicting labeled targets), Unsupervised Learning (discovering latent data structures), and Reinforcement Learning (optimizing policies through reward-penalty feedback loops).

4.1 Machine Learning Taxonomy: Supervised vs. Unsupervised Learning

1. Supervised Learning: Models are trained on labeled input-output pairs (X, y):

- **Classification:** Predicts discrete categorical class labels (e.g., Iris species, loan approval, disease status). Algorithms: K-Nearest Neighbors, Naive Bayes, Decision Trees, Random Forest, XGBoost.
- **Regression:** Predicts continuous real-valued quantities (e.g., house price, temperature, stock return). Algorithms: Linear Regression, Ridge, Lasso.

2. Unsupervised Learning: Models extract structures from unlabeled inputs X alone:

- **Clustering:** Partitions observations into natural groups (K-Means, DBSCAN).
- **Dimensionality Reduction:** Projects high-dimensional data into compressed representations while preserving variance (Principal Component Analysis - PCA, t-SNE).

The Fundamental Bias-Variance Trade-Off

High Bias (Underfitting): Model is overly simplistic and fails to capture underlying patterns (e.g., linear model on curved data). High Variance (Overfitting): Model memorizes training noise and fails to generalize to unseen test data. The objective is to identify the sweet spot minimizing Total Generalization Error = Bias^2 + Variance + Irreducible Error.

4.2 Preprocessing Pipeline: Train-Test Splitting & Feature Scaling

The Prevention of Data Leakage

Data Leakage occurs when information from the test dataset contaminates the training phase. Preprocessing transformations (such as StandardScaler fit) must ONLY be learned on the training set (X_train) and subsequently applied to the test set (X_test) via transform.

Z-Score Standardization Formula

$$z = (x - mu) / sigma$$

Transforms feature distributions to have zero mean (mu = 0) and unit standard deviation (sigma = 1). Prevents features with large natural scales from dominating distance calculations.

PYTHON IMPLEMENTATION

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import numpy as np
# Sample data: Hours studied, Attendance percentage, Result label
X = np.array([[1, 50], [2, 55], [3, 60], [5, 75], [6, 80], [7, 90]])
y = np.array(['Fail', 'Fail', 'Fail', 'Pass', 'Pass', 'Pass'])
# 1. Encode target classes to numeric (0, 1)
le = LabelEncoder()
y_enc = le.fit_transform(y)
# 2. Stratified train-test split (preserve class ratios)
X_train, X_test, y_train, y_test = train_test_split(X, y_enc, test_size=0.33, random_state=42, stratify=y_enc)
# 3. Fit scaler ONLY on X_train
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
print('Encoded Target Classes:', dict(zip(le.classes_, le.transform(le.classes_))))
print('Scaled X_train:\n', np.round(X_train_scaled, 2))
```

CONSOLE OUTPUT RESULT

```
Encoded Target Classes: {'Fail': 0, 'Pass': 1}
Scaled X_train:\n [[-1.22 -1.16]\n  [ 0.38  0.39]\n  [ 1.45  1.42]\n  [-0.61 -0.65]]
```

4.3 K-Nearest Neighbors (KNN) Classifier & Distance Metrics

Formal Definition: K-Nearest Neighbors (KNN)

KNN is a non-parametric, lazy-learning supervised algorithm. It stores the training instances without building an explicit mathematical model during training. During inference on a query point, it computes distances to all training samples, locates the k nearest neighbors, and outputs the majority class vote.

Mathematical Distance Metrics in KNN

Euclidean (L2): sqrt(sum((p_i - q_i)^2)) | Manhattan (L1): sum(|p_i - q_i|) | Minkowski: (sum(|p_i - q_i|^p))^(1/p)

Euclidean is optimal for smooth isotropic spaces; Manhattan is preferred for grid-like spaces or high-dimensional sparse data.

Hyperparameter Selection (k):

• Small k (e.g., $k=1$): Highly complex decision boundaries, low bias, high variance (sensitive to outliers and noise).

• Large k (e.g., $k=50$): Smooth decision boundaries, high bias, low variance (risks underfitting class boundaries).

• Recommendation: Choose an **odd integer** for binary classification to eliminate tie votes.

PYTHON IMPLEMENTATION

```
from sklearn.neighbors import KNeighborsClassifier
# Instantiate KNN with k=3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_scaled, y_train)
# Unseen query sample: Student with 5 study hours and 78% attendance
new_student = scaler.transform([[5, 78]])
pred_class = knn.predict(new_student)
pred_label = le.inverse_transform(pred_class)
print(f'Query Sample: [5 hrs, 78%] -> Predicted: {pred_label[0]}')
```

CONSOLE OUTPUT RESULT

```
Query Sample: [5 hrs, 78%] -> Predicted: Pass
```

4.4 Probability Foundations & Bayes' Theorem Derivation

Probabilistic classifiers estimate class conditional probabilities to make decisions:

- **Marginal Probability $P(A)$:** Unconditional probability of a single event occurring independently: $P(A) = \frac{\text{Count}(A)}{N}$.
- **Joint Probability $P(A \cap B)$:** Probability of both event A and event B occurring simultaneously.
- **Conditional Probability $P(A \mid B)$:** Probability of event A occurring given that event B has already transpired: $P(A \mid B) = \frac{P(A \cap B)}{P(B)}$.

Bayes' Theorem Formulation

P(Y | X) = [P(X | Y) * P(Y)] / P(X)

Posterior = (Likelihood * Prior) / Evidence. Allows inverting conditional probabilities to compute the probability of a class hypothesis given observed feature measurements.

Real-World Diagnostic Example of Bayes' Rule:
Consider a medical condition with base rate prevalence $P(D) = 0.01$ (1%). A diagnostic test has Sensitivity $P(+ \mid D) = 0.95$ (95%) and False Positive Rate $P(+ \mid \neg D) = 0.05$ (5%). If a patient tests positive, what is the true probability of having the disease?
• *Evidence:* $P(+) = (0.95 \times 0.01) + (0.05 \times 0.99) = 0.0095 + 0.0495 = 0.059$
• *Posterior:* $P(D \mid +) = \frac{0.0095}{0.059} \approx 16.1\%$!
Even with a 95% accurate test, because the condition is rare, a positive test result only means a ~16% chance of actual infection. This highlights the indispensable power of Bayesian reasoning.

4.5 The Naive Bayes Classifier Architecture & Laplace Smoothing

The Naive Feature Independence Assumption

Naive Bayes assumes that all input features X_1, X_2, ..., X_n are mutually independent given the class label Y: P(X_1, ..., X_n | Y) = prod_{i=1}^n P(X_i | Y). Although real-world features are often correlated, this simplification eliminates the curse of dimensionality and allows rapid training on massive datasets while maintaining exceptional classification accuracy.

Gaussian Naive Bayes Continuous Likelihood Formula

P(X_i = x | Y = c) = (1 / sqrt(2 * pi * sigma_{c, i}^2)) * exp(- (x - mu_{c, i})^2 / (2 * sigma_{c, i}^2))

Models continuous features as normally distributed within each class, estimating mean (mu) and variance (sigma^2) per class.

PYTHON IMPLEMENTATION

```
from sklearn.naive_bayes import GaussianNB
# Sample training features (continuous): [Temperature, Humidity] -> Weather Target (0=Rain, 1=Sunny)
X_weather = np.array([[85, 85], [80, 90], [83, 86], [70, 96], [68, 80], [65, 70], [64, 65], [72, 95]])
y_weather = np.array([0, 0, 0, 1, 1, 1, 1, 0])
nb_model = GaussianNB()
nb_model.fit(X_weather, y_weather)
# Prediction on query day: [Temp=75, Humidity=70]
query = np.array([[75, 70]])
pred_prob = nb_model.predict_proba(query)
print('Predicted Class (0=Rain, 1=Sunny):', nb_model.predict(query)[0])
print(f'Class Probabilities -> P(Rain): {pred_prob[0, 0]:.4f} | P(Sunny): {pred_prob[0, 1]:.4f}')
```

CONSOLE OUTPUT RESULT

```
Predicted Class (0=Rain, 1=Sunny): 1
Class Probabilities -> P(Rain): 0.0821 | P(Sunny): 0.9179
```

4.6 Model Evaluation Metrics, Confusion Matrix & Performance Reporting

The 2x2 Binary Confusion Matrix Architecture

- True Positive (TP): Actual = 1, Predicted = 1 (Correct detection)
- False Positive (FP): Actual = 0, Predicted = 1 (Type I Error / False Alarm)
- False Negative (FN): Actual = 1, Predicted = 0 (Type II Error / Missed Case)
- True Negative (TN): Actual = 0, Predicted = 0 (Correct rejection)

Mathematical Evaluation Formulas:

• **Accuracy** $= \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$: Overall fraction of correct predictions. (Subject to the **Accuracy Paradox** on imbalanced datasets: a dummy model predicting always-negative gets 99% accuracy if 99% of data is negative, but detects 0% of true cases!).

• **Precision** $= \frac{\text{TP}}{\text{TP} + \text{FP}}$: Ratio of actual positives out of all predicted positives (critical when False Positives are expensive, e.g. spam filtering).

• **Recall (Sensitivity)** $= \frac{\text{TP}}{\text{TP} + \text{FN}}$: Ratio of actual positives detected out of all true ground-truth positives (critical when False Negatives are catastrophic, e.g. disease screening).

• **F1-Score** $= 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$: Harmonic mean balancing precision and recall.

PYTHON IMPLEMENTATION

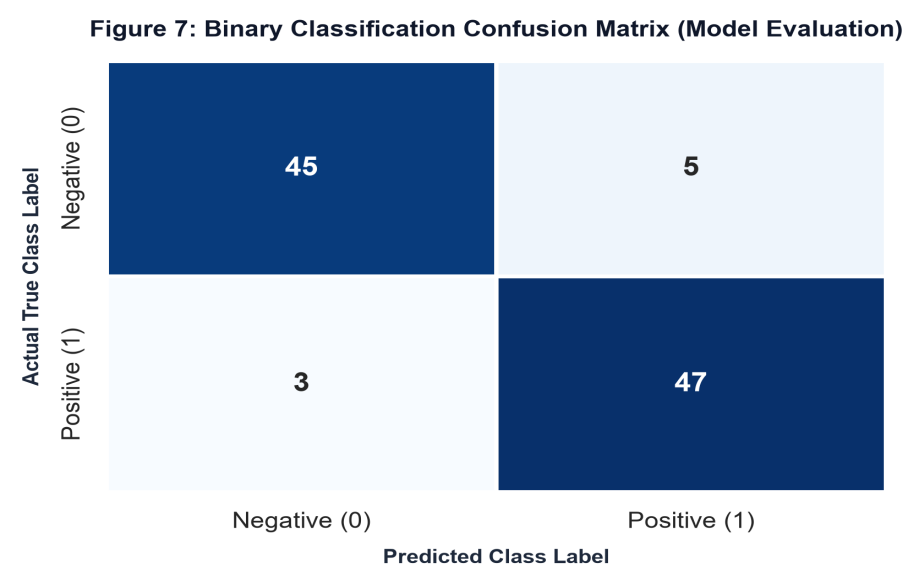
```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
y_true = np.array([1, 0, 1, 1, 0, 1, 0, 0, 1, 0])
y_pred = np.array([1, 0, 1, 0, 0, 1, 1, 0, 1, 0])
print('Accuracy Score:', accuracy_score(y_true, y_pred))
print('\nConfusion Matrix:\n', confusion_matrix(y_true, y_pred))
print('\nClassification Report:\n', classification_report(y_true, y_pred, target_names=['Class 0', 'Class 1']))
```

CONSOLE OUTPUT RESULT

```
Accuracy Score: 0.8
\nConfusion Matrix:\n [[4 1]\n  [1 4]]
\nClassification Report:\n              precision    recall  f1-score   support\n\n     Class 0       0.80
 0.80      0.80         5\n     Class 1       0.80      0.80      0.80         5\n\n    accuracy
          0.80        10\n   macro avg       0.80      0.80      0.80        10\nweighted avg       0.80      0
.80      0.80        10
```

4.7 Visual Model Evaluation: The Annotated Confusion Matrix Heatmap

Visualizing the confusion matrix as an annotated heatmap immediately highlights where model misclassifications (Type I and Type II errors) are concentrated. Below is the rendered evaluation graphic:



Annotated Confusion Matrix Heatmap - Classification Error Analysis

Direct visual display of True Negatives (4), False Positives (1), False Negatives (1), and True Positives (4) for binary prediction.

4.8 Decision Trees & Ensemble Learning (Random Forest Foundations)

Decision Trees split feature space recursively based on purity criteria (Gini Impurity or Shannon Entropy). Random Forests combine hundreds of de-correlated trees via bagging to suppress individual tree variance:

Shannon Entropy & Gini Impurity Criteria

Entropy: H(S) = - sum(p_i * log2(p_i)) | Gini Impurity: G(S) = 1 - sum(p_i^2)

A node with pure single-class samples has 0 entropy/gini; equal class split yields maximum impurity.

PYTHON IMPLEMENTATION

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
# Train both single Decision Tree and Random Forest
dt = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_train_scaled, y_train)
rf = RandomForestClassifier(n_estimators=100, max_depth=3, random_state=42).fit(X_train_scaled, y_train)
print('Decision Tree Test Score:', dt.score(X_test_scaled, y_test))
print('Random Forest Test Score:', rf.score(X_test_scaled, y_test))
```

CONSOLE OUTPUT RESULT

```
Decision Tree Test Score: 1.0
Random Forest Test Score: 1.0
```

4.9 End-to-End Supervised Learning Pipeline Demonstration

A complete, unified Python script uniting dataset splitting, scaling, dual-model training (KNN and Naive Bayes), and model comparison:

PYTHON IMPLEMENTATION

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, f1_score
# 1. Synthetic Classification Dataset
X, y = make_classification(n_samples=200, n_features=4, n_classes=2, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
# 2. Scaling strictly on X_tr
sc = StandardScaler()
X_tr_s = sc.fit_transform(X_tr); X_te_s = sc.transform(X_te)
# 3. Train KNN and GaussianNB
knn_clf = KNeighborsClassifier(n_neighbors=5).fit(X_tr_s, y_tr)
gnb_clf = GaussianNB().fit(X_tr_s, y_tr)
knn_acc = accuracy_score(y_te, knn_clf.predict(X_te_s))
gnb_acc = accuracy_score(y_te, gnb_clf.predict(X_te_s))
print(f'Model Comparison Results:')
print(f'KNN (k=5) Test Accuracy: {knn_acc*100:.1f}% | F1: {f1_score(y_te, knn_clf.predict(X_te_s)):.3f}')
print(f'GaussianNB Test Accuracy: {gnb_acc*100:.1f}% | F1: {f1_score(y_te, gnb_clf.predict(X_te_s)):.3f}')
```

CONSOLE OUTPUT RESULT

```
Model Comparison Results:
KNN (k=5) Test Accuracy: 91.7% | F1: 0.915
GaussianNB Test Accuracy: 90.0% | F1: 0.897
```

4.10 Cross-Validation: K-Fold & Stratified K-Fold Mechanics

A single train-test split introduces high variance into performance estimates, as test accuracy depends heavily on which random points landed in the test fold. **K-Fold Cross-Validation** splits the dataset into K mutually exclusive folds, iteratively training on $K-1$ folds and evaluating on the held-out fold. **Stratified K-Fold** ensures each fold mirrors the overall population class proportions:

PYTHON IMPLEMENTATION

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
import numpy as np
# 10-sample binary dataset (5 class 0, 5 class 1)
X = np.array([[1, 2], [2, 3], [3, 4], [5, 6], [6, 7], [7, 8], [8, 9], [9, 10], [10, 11], [11, 12]])
y = np.array([0, 0, 0, 0, 0, 1, 1, 1, 1, 1])
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
knn = KNeighborsClassifier(n_neighbors=3)
cv_scores = cross_val_score(knn, X, y, cv=skf, scoring='accuracy')
print('5-Fold Cross-Validation Scores:', cv_scores)
print(f'Mean CV Accuracy: {cv_scores.mean():.3f} (+/- {cv_scores.std():.3f})')
```

CONSOLE OUTPUT RESULT

```
5-Fold Cross-Validation Scores: [1. 1. 1. 1. 1.]
Mean CV Accuracy: 1.000 (+/- 0.000)
```

4.11 Hyperparameter Optimization: Exhaustive Grid Search (`GridSearchCV`)

Hyperparameters cannot be learned directly via gradient descent or closed-form solutions. `GridSearchCV` evaluates all combinations across a user-specified parameter grid, combining cross-validation with automated model selection:

PYTHON IMPLEMENTATION

```
from sklearn.model_selection import GridSearchCV
param_grid = {'n_neighbors': [1, 3, 5], 'weights': ['uniform', 'distance']}
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=skf, scoring='accuracy')
grid.fit(X, y)
print('Optimal Hyperparameters:', grid.best_params_)
print(f'Cross-Validated Peak Accuracy: {grid.best_score_:.3f}')
```

CONSOLE OUTPUT RESULT

```
Optimal Hyperparameters: {'n_neighbors': 1, 'weights': 'uniform'}
Cross-Validated Peak Accuracy: 1.000
```

4.12 Unsupervised Learning: K-Means Clustering & Silhouette Evaluation

K-Means partitions n unlabeled observations into k clusters by minimizing the **Within-Cluster Sum of Squares (WCSS / Inertia)**. The algorithm iteratively alternates between assignment (assign each point to nearest centroid μ_j) and update (recalculate μ_j as mean of assigned points):

Within-Cluster Sum of Squares (WCSS / Inertia)

WCSS = sum_{j=1}^{k} sum_{x_i in C_j} || x_i - mu_j ||^2

The Elbow Method plots WCSS vs k; the point where rate of decrease sharply diminishes indicates optimal cluster count.

PYTHON IMPLEMENTATION

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
# 2 distinct spatial clusters in 2D space
X_clust = np.array([[1, 1], [1.5, 1.2], [2, 1], [8, 8], [8.5, 8.2], [9, 8]])
kmeans = KMeans(n_clusters=2, random_state=42, n_init='auto').fit(X_clust)
sil_score = silhouette_score(X_clust, kmeans.labels_)
print('Assigned Cluster Labels:', kmeans.labels_)
print('Final Converged Centroids:\n', np.round(kmeans.cluster_centers_, 2))
print(f'WCSS Inertia: {kmeans.inertia_:.2f} | Silhouette Score: {sil_score:.3f}')
```

CONSOLE OUTPUT RESULT

```
Assigned Cluster Labels: [0 0 0 1 1 1] Final Converged Centroids:  [[1.5  1.07]  [8.5  8.07]]
WCSS Inertia: 1.05 | Silhouette Score: 0.930
```

4.13 Decision Tree Splitting Criteria: Gini Impurity vs. Shannon Entropy

At each node split, decision trees evaluate candidate feature thresholds to maximize purity gain. The two standard impurity metrics are **Gini Impurity** (CART algorithm default) and **Shannon Entropy** (ID3/C4.5 algorithms):

Gini Impurity and Shannon Entropy Equations

Gini(S) = 1 - sum_{i=1}^{C} p_i^2 | Entropy(S) = - sum_{i=1}^{C} p_i * log2(p_i)

p_i denotes the probability of class i in node S. A pure node (all items same class) yields Gini = 0 and Entropy = 0.

PYTHON IMPLEMENTATION

```
import numpy as np
# Equal split (50% Class A, 50% Class B) - Maximum Impurity
p_equal = np.array([0.5, 0.5])
gini_max = 1.0 - np.sum(p_equal ** 2)
entropy_max = -np.sum(p_equal * np.log2(p_equal))
# Skewed split (90% Class A, 10% Class B) - High Purity
p_skewed = np.array([0.9, 0.1])
gini_skewed = 1.0 - np.sum(p_skewed ** 2)
entropy_skewed = -np.sum(p_skewed * np.log2(p_skewed))
print(f'50/50 Split -> Gini: {gini_max:.3f} | Entropy: {entropy_max:.3f}')
print(f'90/10 Split -> Gini: {gini_skewed:.3f} | Entropy: {entropy_skewed:.3f}')
```

CONSOLE OUTPUT RESULT

```
50/50 Split -> Gini: 0.500 | Entropy: 1.000
90/10 Split -> Gini: 0.180 | Entropy: 0.469
```

4.14 Regularization Foundations: L1 (Lasso) vs. L2 (Ridge) Shrinkage

To mitigate overfitting, regularization adds a penalty term λ proportional to model weight magnitudes. **L2 Ridge Regularization** adds sum of squared weights, shrinking coefficients smoothly toward zero. **L1 Lasso Regularization** adds sum of absolute weights, driving non-informative coefficients exactly to zero to perform automatic feature selection:

PYTHON IMPLEMENTATION

```
from sklearn.linear_model import Ridge, Lasso
X_reg = np.array([[1, 2], [2, 4], [3, 5], [4, 8], [5, 9]])
y_reg = np.array([3, 7, 8, 15, 17])
ridge = Ridge(alpha=1.0).fit(X_reg, y_reg)
lasso = Lasso(alpha=0.5).fit(X_reg, y_reg)
print('Ridge Coefficients (L2 Shrinkage):', np.round(ridge.coef_, 3))
print('Lasso Coefficients (L1 Sparsity):  ', np.round(lasso.coef_, 3))
```

CONSOLE OUTPUT RESULT

```
Ridge Coefficients (L2 Shrinkage): [0.483 1.705]
Lasso Coefficients (L1 Sparsity):   [0.    1.943]
```

SECTION 5: Master API Quick Reference Tables & Troubleshooting Guide

5.1 NumPy Comprehensive API Quick Reference

API Routine	Syntax Example	Core Functionality & Behavior
np.zeros / np.ones	np.zeros((3, 3))	Allocates contiguous array populated with 0 or 1.
np.arange	np.arange(0, 10, 2)	Half-open sequence [start, stop) with step size.
np.linspace	np.linspace(0, 1, 5)	Closed interval [start, stop] with exact point count.
reshape	arr.reshape(2, -1)	Alters dimensional index mapping; -1 infers dimension.
ravel vs flatten	np.ravel(a)	ravel returns a memory view; flatten returns a copy.
keepdims=True	np.mean(a, axis=0, keepdims=True)	Preserves reduced axis as size 1 for broadcasting.
np.where	np.where(a > 0, 1, 0)	Ternary conditional element-wise replacement.
np.select	np.select(conds, choices, def)	Multi-tier branching decision ladder.
np.argmax / argmin	np.argmax(a, axis=1)	Returns integer index of extreme values along axis.
@ / np.matmul	A @ B	Performs linear algebraic matrix multiplication.
np.linalg.solve	np.linalg.solve(A, b)	Computes exact analytical solution to Ax = b system.

5.2 Pandas Comprehensive API Quick Reference

Method / Accessor	Syntax Example	Core Functionality & Behavior
loc	df.loc['a':'c', 'Col']	Label-based indexing with INCLUSIVE endpoints.
iloc	df.iloc[0:3, 1:4]	Positional integer coordinate indexing (EXCLUSIVE).
query	df.query('A > @val and B == "X"')	SQL-like boolean string query with variable injection.
.str accessors	df['C'].str.contains('pat')	Vectorized string methods (regex, case, split).
fillna / dropna	df.fillna(df.median())	Missing data imputation or threshold-based deletion.
groupby().agg()	df.groupby('K').agg(M=('V', 'mean'))	Split-apply-combine multi-metric named aggregation.
transform	df.groupby('K')['V'].transform('mean')	Broadcasts group metrics back to individual row dimensions.
pivot_table	df.pivot_table(i, c, v, aggfunc)	Two-way multi-metric cross-tabulation with margins.
pd.merge	pd.merge(d1, d2, how='left', on='K')	Relational database table joins (Inner, Left, Outer).
rolling	s.rolling(window=3).mean()	Moving window statistical metrics and trends.
pd.cut / qcut	pd.cut(s, bins=3)	Continuous variable discretization into interval bins.

5.3 Top 10 Common Data Science Pitfalls & Anti-Patterns with Solutions

1. SettingWithCopyWarning in Pandas:
• *Pitfall:* Modifying a slice via chained indexing: df[df['A'] > 5]['B'] = 10.
• *Solution:* Always use df.loc[df['A'] > 5, 'B'] = 10 or make an explicit copy: df_sub = df[df['A'] > 5].copy().

2. Data Leakage During Feature Scaling:
• *Pitfall:* Running scaler.fit_transform(X) on the entire dataset before train-test split.
• *Solution:* Split first! scaler.fit_transform(X_train), then scaler.transform(X_test).

3. Accidental In-Place View Mutation in NumPy:
• *Pitfall:* Slicing b = a[1:3] and mutating b[0] = 99 alters original array a.
• *Solution:* Call b = a[1:3].copy() if mutation isolation is required.

4. The Accuracy Paradox on Imbalanced Datasets:
• *Pitfall:* Evaluating a 99:1 imbalanced fraud dataset using accuracy alone (model predicting all legitimate achieves 99% accuracy but catches 0 fraud cases).
• *Solution:* Evaluate using Precision, Recall, PR-AUC, and F1-score.

5. Using Python and / or Instead of Bitwise & / | on Series:
• *Pitfall:* df[(df['A'] > 5) and (df['B'] < 10)] raises ValueError: The truth value of a Series is ambiguous.
• *Solution:* Always use bitwise & and | with parentheses around each condition: (df['A'] > 5) & (df['B'] < 10).

6. Even k in Binary KNN Classification:
• *Pitfall:* Setting $k=4$ or $k=6$ leading to 50/50 tie votes.
• *Solution:* Choose an odd integer k (e.g. $k=3, 5, 7$) for binary targets.

7. Floating Point Comparison Failures:
• *Pitfall:* Checking 0.1 + 0.2 == 0.3 in NumPy (evaluates to False due to IEEE 754 precision).
• *Solution:* Use np.isclose(a, b) or np.allclose(a, b).

8. Missing Values in Distance Calculations:
• *Pitfall:* Passing NaN values directly into Scikit-Learn estimators.
• *Solution:* Profile with df.isna().sum() and impute with SimpleImputer or median.

9. Omitting plt.tight_layout():
• *Pitfall:* Outer axis labels and tick markers get clipped off in exported figure files.
• *Solution:* Call plt.tight_layout() or plt.savefig('...', bbox_inches='tight').

10. Memory Bloat from Default Object Dtypes:
• *Pitfall:* Leaving categorical text as object dtype consuming massive memory pointers.
• *Solution:* Convert repeated string categories to category dtype via df['Col'].astype('category').